

# ELECTRONIC PRINCIPLES AND DEVICES

---

## Course Content

Department of Electronics and Communication.

# ELECTRONIC PRINCIPLES AND DEVICES

---

## Unit 4 –DIGITAL ELECTRONICS

# ELECTRONIC PRINCIPLES AND DEVICES

---

## Syllabus:

- Number Systems – binary and hexadecimal
- Binary Addition and Subtraction, 2's complement subtraction
- Boolean Algebra, Logic gates, Basic Theorems and Properties of Boolean Algebra
- Boolean Functions, Canonical and Standard Form, other Logical Operations.
- Combinational Logic Circuits: Half Adder and Full adder
- Sequential Circuits: RS, D, T, JK Flip-Flops, SISO Register, 3 Bit Asynchronous Up/Down counters.

**Text book:** “Digital Design with an Introduction to Verilog HDL”, M Morris Mano, Michale D Ciletti, Pearson, 5th Edition, 2013.

# ELECTRONIC PRINCIPLES AND DEVICES

---



## Introduction

- Digital electronics is the branch of electronics that deals with the digital systems which processes the data/information in the form of binary(0s and 1s) numbers.
- These binary values correspond to two voltage levels, typically representing "low" (0) and "high" (1).

## Number System

- Number system a mathematical notation for representing numbers using digits or other symbols in a consistent manner.
- Each number system is characterized by its **base** or **radix**, which determines the number of symbols used.
- **Base** (or **Radix**) refers to the number of unique digits or symbols used in a number system.
  - For example: The **Decimal number system** has a base of 10, using digits 0 to 9.  
The **Binary number system** has a base of 2, using digits 0 and 1.  
The **Hexadecimal number system** has a base of 16, using digits 0 to 9 and letters A to F.

# ELECTRONIC PRINCIPLES AND DEVICES

---

## Binary Number System

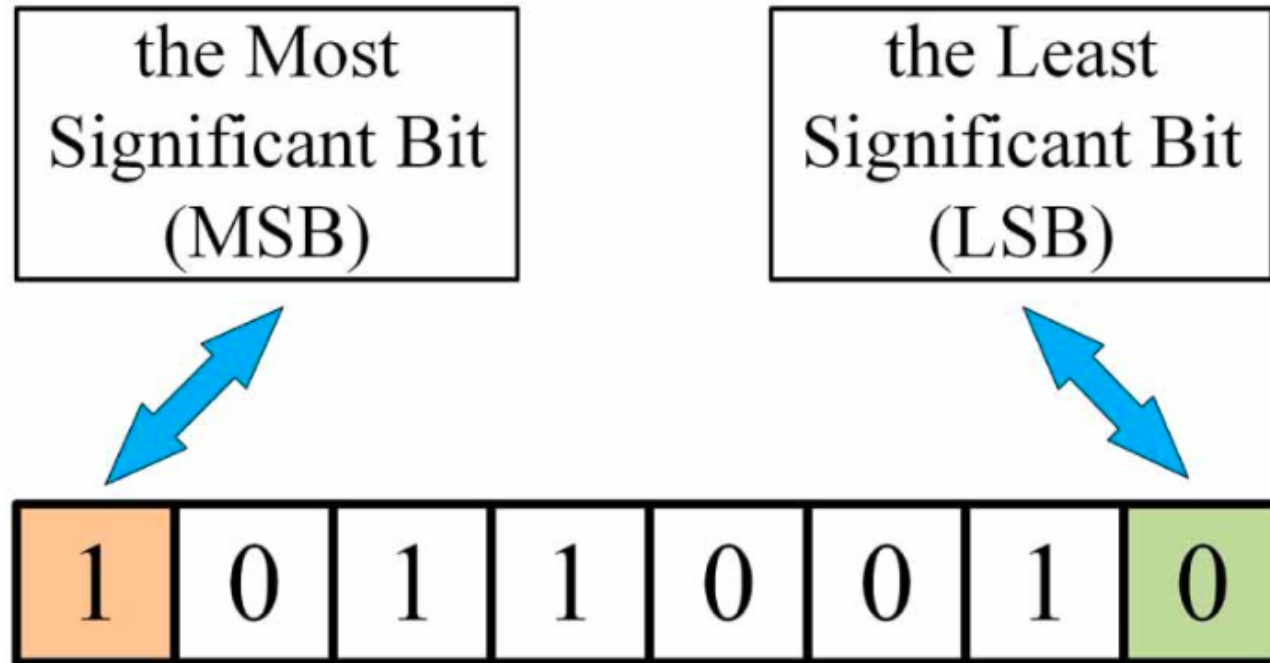
### Key features

- The **binary number system** is a base-2 system that uses only two digits: **0** and **1**.
- Each digit in a binary number is called a **bit** (short for "binary digit").
- **Place Value:** Each bit in a binary number represents a specific power of 2, starting from  $2^0$  (rightmost bit) and increasing to  $2^1, 2^2, 2^3$ , etc., as you move leftward.
- **Representation of Numbers:**  
Any numerical value can be represented in binary form using combinations of 0s and 1s.  
The number of **bits** in a binary number determines the total number of unique combinations or values that can be represented.

If  $n$  is the number of bits, the maximum number values that can be represented using  $n$  bits equals  $2^n$

# ELECTRONIC PRINCIPLES AND DEVICES

- In a binary number, the rightmost bit is called the **Least Significant Bit (LSB)** and the left most bit is the **Most Significant Bit (MSB)**.



The place value of the LSB is  $2^0$  and the next digits to the left gets the place value in terms of increasing powers of 2.

In the above number, the place value of MSB is  $2^7$ .

# ELECTRONIC PRINCIPLES AND DEVICES

2 bit binary

| Representation | Range |
|----------------|-------|
| 00             | 0     |
| 01             | 1     |
| 10             | 2     |
| 11             | 3     |

3 bit binary

| Representation | Range |
|----------------|-------|
| 000            | 0     |
| 001            | 1     |
| 010            | 2     |
| 011            | 3     |
| 100            | 4     |
| 101            | 5     |
| 110            | 6     |
| 111            | 7     |

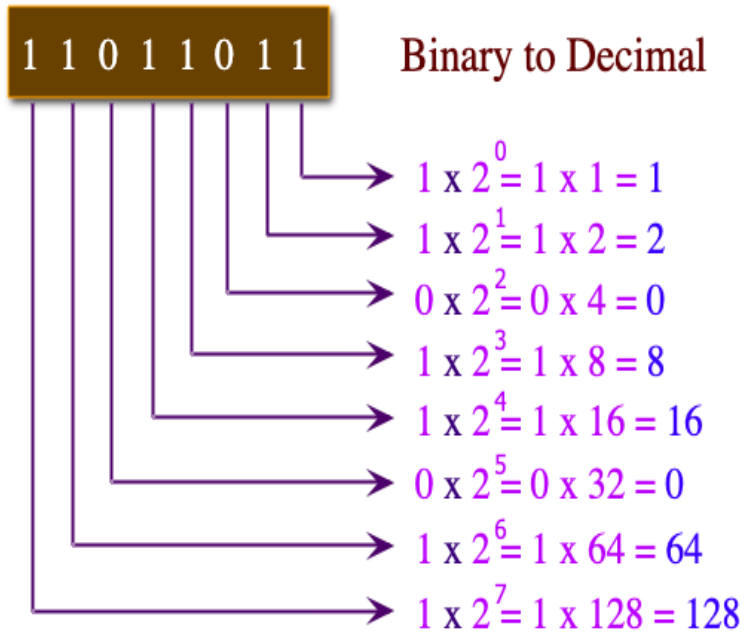
4 bit binary

| Representation | Range |
|----------------|-------|
| 0000           | 0     |
| 0001           | 1     |
| 0010           | 2     |
| 0011           | 3     |
| 0100           | 4     |
| 0101           | 5     |
| 0110           | 6     |
| 0111           | 7     |
| 1000           | 8     |
| 1001           | 9     |
| 1010           | 10    |
| 1011           | 11    |
| 1100           | 12    |
| 1101           | 13    |
| 1110           | 14    |
| 1111           | 15    |

# ELECTRONIC PRINCIPLES AND DEVICES

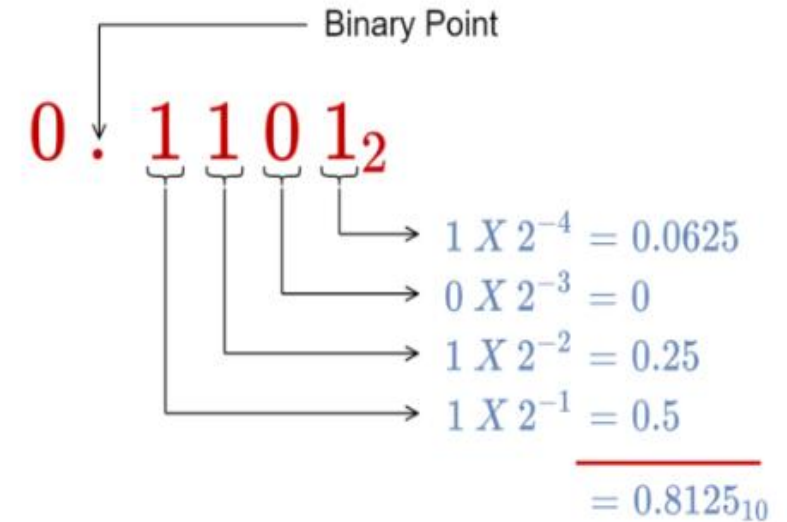
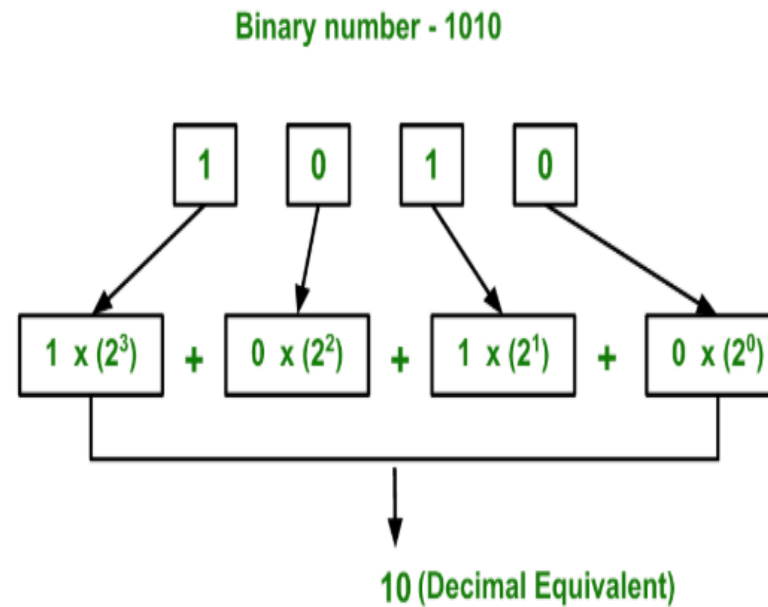
## Binary to Decimal Conversion:

- Multiply each binary digit by the value of its position or place value (power of 2).
- Add all the products together to get the decimal equivalent



$$1 + 2 + 8 + 16 + 64 + 128 = 219$$

$$(11011011)_2 = (219)_{10}$$



# ELECTRONIC PRINCIPLES AND DEVICES

Convert the following binary numbers to its decimal equivalent

a)

**1011** Starting from LSB,  
 $1 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 1 \times 2^3$   
 $= 1 + 2 + 0 + 8 = 11$

$$(1011)_2 = (11)_{10}$$

b) **11010**

$$0 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 1 \times 2^3 + 1 \times 2^4$$
$$= 0 + 2 + 0 + 8 + 16 = 26$$

$$(11010)_2 = (26)_{10}$$

c) **1111101**

$$(1111101)_2 = (125)_{10}$$

d) **101.01011**

$$1 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 = 1 + 0 + 4 = 5$$

$$(.01011)_2 = 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-5}$$
$$= 0.34375$$

$$(101.01011)_2 = (5.34375)_{10}$$

# ELECTRONIC PRINCIPLES AND DEVICES

## Decimal to Binary Conversion

- Divide the Decimal Number by 2.
- Record the quotient and the remainder.
- Repeat the division by 2 until the quotient is 0.
- The last remainder represents the MSB and the first remainder represents the LSB. The remainders represent the equivalent binary number

|   |      |   |       |
|---|------|---|-------|
| 2 | 4215 |   |       |
| 2 | 2107 | 1 | ← LSB |
| 2 | 1053 | 1 |       |
| 2 | 526  | 1 |       |
| 2 | 263  | 0 |       |
| 2 | 131  | 1 |       |
| 2 | 65   | 1 |       |
| 2 | 32   | 1 |       |
| 2 | 16   | 0 |       |
| 2 | 8    | 0 |       |
| 2 | 4    | 0 |       |
| 2 | 2    | 0 |       |
| 2 | 1    | 0 |       |
|   | 0    | 1 | ← MSB |

# Decimal to Binary Conversion

$(27)_{10} = (11011)_2$

| 2 | 27 | Remainder |
|---|----|-----------|
| 2 | 13 | 1         |
| 2 | 6  | 1         |
| 2 | 3  | 0         |
| 2 | 1  | 1         |
|   | 0  | 1         |

# ELECTRONIC PRINCIPLES AND DEVICES

Convert  $(0.6875)_{10}$  to binary.

|                     | Integer |   | Fraction |
|---------------------|---------|---|----------|
| $0.6875 \times 2 =$ | 1       | + | 0.3750   |
| $0.3750 \times 2 =$ | 0       | + | 0.7500   |
| $0.7500 \times 2 =$ | 1       | + | 0.5000   |
| $0.5000 \times 2 =$ | 1       | + | 0.0000   |

$$(0.6875)_{10} = (0.1011)_2$$

Convert the decimal number 41 to binary.

|          | Integer<br>Quotient |   | Remainder     | Coefficient |
|----------|---------------------|---|---------------|-------------|
| $41/2 =$ | 20                  | + | $\frac{1}{2}$ | $a_0 = 1$   |
| $20/2 =$ | 10                  | + | 0             | $a_1 = 0$   |
| $10/2 =$ | 5                   | + | 0             | $a_2 = 0$   |
| $5/2 =$  | 2                   | + | $\frac{1}{2}$ | $a_3 = 1$   |
| $2/2 =$  | 1                   | + | 0             | $a_4 = 0$   |
| $1/2 =$  | 0                   | + | $\frac{1}{2}$ | $a_5 = 1$   |

$$(41)_{10} = (101001)_2$$

# ELECTRONIC PRINCIPLES AND DEVICES

## Hexadecimal Number System

The **hexadecimal (base-16)** number system is a positional numeral system that uses **16 distinct symbols** to represent values.

### Symbols Used:

- **Digits:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- **Letters:** A, B, C, D, E, F  
(where A = 10, B = 11, C = 12, D = 13, E = 14, F = 15 in decimal).

### • Place Values:

Each digit in a hexadecimal number represents a power of 16, starting from  $16^0$  at the rightmost position

Ex: 1A316, 2AC

| Hexadecimal | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A  | B  | C  | D  | E  | F  |
|-------------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| Decimal     | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |

# ELECTRONIC PRINCIPLES AND DEVICES



## Hexadecimal to Decimal Conversion

- Convert each hexadecimal bit into its decimal equivalent.
- Multiply each bit by the value of its position or place value (power of 16).
- Add all the products together to get the decimal equivalent

2F36

$$6 \times 16^0 + 3 \times 16^1 + 15 \times 16^2 + 2 \times 16^3$$

$$= 8192 + 3840 + 48 + 6$$
$$= 12086$$

(B65F)<sub>16</sub>

$$(B65F)_{16} = 11 \times 16^3 + 6 \times 16^2 + 5 \times 16^1 + 15 \times 16^0 = (46,687)_{10}$$

54.D2<sub>16</sub>

$$= 5 \cdot 16^1 + 4 \cdot 16^0 + D \cdot 16^{-1} + 2 \cdot 16^{-2}$$
$$= 5 \cdot 16^1 + 4 \cdot 16^0 + 13 \cdot 16^{-1} + 2 \cdot 16^{-2}$$
$$= 80 + 4 + 0.8125 + 0.0078125$$
$$= 84.8203125$$

| Hexadecimal | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A  | B  | C  | D  | E  | F  |
|-------------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| Decimal     | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |

# ELECTRONIC PRINCIPLES AND DEVICES

## Decimal to Hexadecimal Conversion

- **Divide the Decimal Number by 16**

Divide the decimal number by 16. The quotient will be used for the next division, and the remainder will be one of the hexadecimal digits.

- **Record the Remainder**

The remainder of each division corresponds to a hexadecimal digit. If the remainder is between 10 and 15, use the hexadecimal letters.

|             |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |
|-------------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| Hexadecimal | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A  | B  | C  | D  | E  | F  |
| Decimal     | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |

- **Repeat the Division**

Divide the quotient by 16 again, and continue recording the remainders until the quotient is 0.

- **Read the Remainders in Reverse Order**

The remainders, when read from last to first, form the hexadecimal representation of the decimal number.

# ELECTRONIC PRINCIPLES AND DEVICES

---



**Example 1: Convert 255 (Decimal) to Hexadecimal**

**Step 1: Divide 255 by 16:**

$$255 \div 16 = 15 \text{ (quotient)} \quad \text{remainder} = 15$$

**Step 2: Divide the quotient (15) by 16:**

$$15 \div 16 = 0 \text{ (quotient)} \quad \text{remainder} = 15$$

**Step 3: The quotient is now 0, so stop dividing.**

**Step 4: Read the remainders in reverse order:** The remainders, from last to first, are F and F.  
Thus, **255 (decimal) = FF (hexadecimal)**.

# ELECTRONIC PRINCIPLES AND DEVICES

## Example 2:

$(374.37)_{10}$

|    |     |   |
|----|-----|---|
| 16 | 374 |   |
| 16 | 23  | 6 |
| 16 | 1   | 7 |
|    | 0   | 1 |

↑

$0.37 \times 16 = 5.92 = 0.92$  with Carry 5  
 $0.92 \times 16 = 14.72 = 0.72$  with Carry 14 (E)  
 $0.72 \times 16 = 11.52 = 0.52$  with Carry 11 (B)  
 $0.52 \times 16 = 8.32 = 0.32$  with Carry 8

$(176)_{16}$        $(0.5EB8)_{16}$

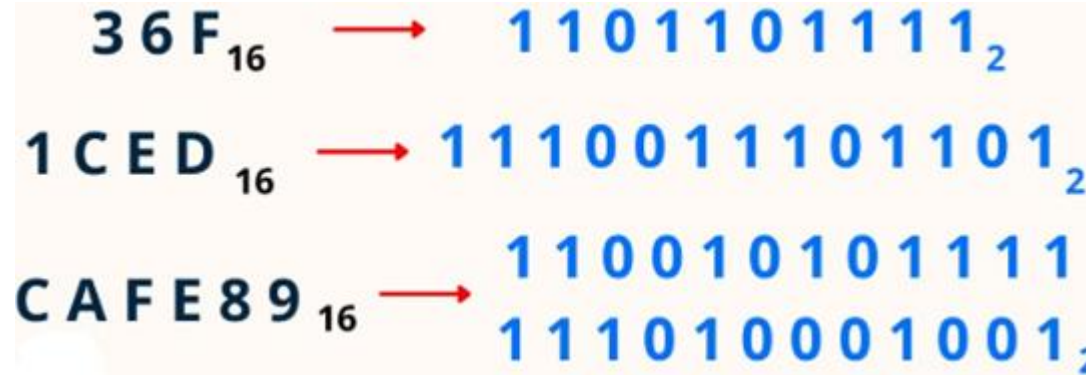
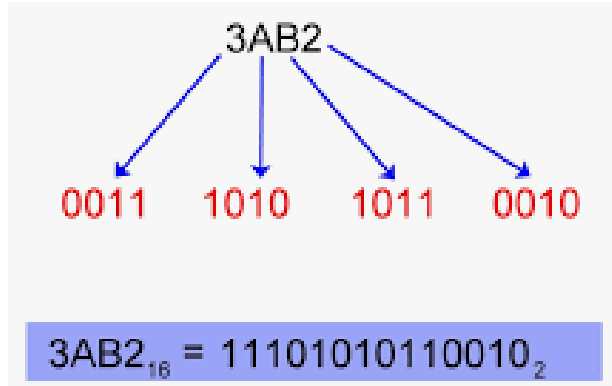
Integer Part      Fraction Part

$(374.37)_{10} = (176.5EB8)_{16}$

# ELECTRONIC PRINCIPLES AND DEVICES

## Hexadecimal to Binary conversion

- Replace each hexadecimal digit with its equivalent 4-bit binary representation



## Binary to Hexadecimal conversion

- Break down the binary number into groups with 4 digits in each group, write the hexadecimal equivalent of each of the groups.
- Combine all the numbers together to get the hexadecimal number.

(100100)<sub>2</sub>  
100100  
00100100  
(24)<sub>16</sub>  
7/4/2025

(1101011.00101)  
1101011.00101  
01101011.00101000  
(6B.28)<sub>16</sub>

# ELECTRONIC PRINCIPLES AND DEVICES

## 1's Complement



- The 1's complement of a binary number is the result of inverting all the bits in the number's binary representation.
- This means swapping all 0s for 1s and vice versa.
- For example, the 1's complement of 1010 is 0101.

In 1's complement representation, the representation of the negative number is different.

For example, if we want to represent -34 in 8-bit 1's complement form, then first write the positive number (+34). And invert all 1s in that number by 0s and 0s by 1s in that number. The corresponding inverted number represents the -34 in 1's complement form. It is also called 1s complement of the number +34.

To represent **-34** in 1's complement form

$$\begin{array}{rcll} +34 & = & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ & & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ -34 & = & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & \text{(1's complement of +34)} \end{array}$$

# ELECTRONIC PRINCIPLES AND DEVICES

## 2's Complement

- 2's complement is the most common method used to represent signed numbers

To find the 2's complement

- Find the 1's complement
- Add 1 to the 1's complement

To represent **-34** in 2's complement form

$$\begin{array}{r} + 34 = 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \\ \quad \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \\ \quad 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \quad (1's \text{ complement of } + 34) \\ \quad + \quad \quad \quad \quad \quad \quad \quad 1 \\ \hline - 34 = 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0 \quad (2's \text{ complement of } + 34) \end{array}$$

# ELECTRONIC PRINCIPLES AND DEVICES

---



- Range of Values for Unsigned Binary Numbers  
In **unsigned binary**, all values are non-negative, starting from 0.

$$\text{Range} = 0 \text{ to } (2^n - 1)$$

For ex: With  $n=1$  bit: no of values is  $2^1=2$  values (0,1)

With  $n=2$  bits: no of values is  $2^2=4$  values (00,01,10,11), 0 to 3 in decimal.

With  $n=3$  bits:  $2^3=8$  values (000-111), 0 to 7 in decimal

- Range of Values for Signed Binary Numbers (2's complement):

$$\text{Range} = -2^{n-1} \text{ to } (2^{n-1} - 1)$$

For ex: With 2 bits:  $2^2=4$  values (-2 to 1), -2,-1,0,1

With 3 bits:  $2^3=8$  values (-4 to 3), -4,-3,-2,-1,0,1,2,3

# ELECTRONIC PRINCIPLES AND DEVICES

## Binary Addition

**A+B=? (Both the numbers are positive)**

(1)  $8 + 5 = ?$

Binary representation of 8 = 00001000

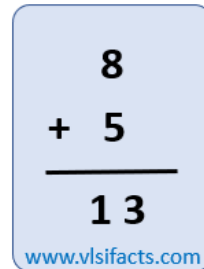
Binary representation of 5 = 00000101

$$\begin{array}{r} 00001000 \\ + 00000101 \\ \hline 00001101 \end{array}$$

Sum: 00001101



Sign bit is zero, so the result is a positive number


$$\begin{array}{r} 8 \\ + 5 \\ \hline 13 \end{array}$$

www.vlsifacts.com

13 => 00001101

(2)  $6 + 13 = ?$

$$\begin{array}{r} + 6 \quad 00000110 \\ +13 \quad 00001101 \\ \hline +19 \quad 00010011 \end{array}$$

# ELECTRONIC PRINCIPLES AND DEVICES

## Binary Subtraction using 2's Complement

---



**A-B=? (One of the numbers is negative)**

**Steps to find A-B=?**

1. At first, find 2's complement of the B (subtrahend).
2. Then add it to the A (minuend).
3. If the final carry over of the sum is 1, then it is dropped and the result is positive. ☐ (A>B)
4. If there is no carry over, then 2's complement of the sum is the final result and it is negative. ☐ (A<B)

# ELECTRONIC PRINCIPLES AND DEVICES

## Binary Subtraction using 2's Complement

(3)  $8 - 5 = ?$

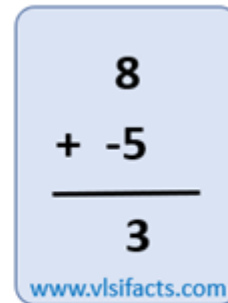
Binary representation of 8 = 00001000

Binary representation of 5 = 00000101

Step-1: 2's complement of  $(00000101)_2$  is  $(11111011)_2$ .

Step-2: Add  $(00001000)_2$  to  $(11111011)_2$ . This is shown below.

$$\begin{array}{r} 00001000 \\ + 11111011 \\ \hline \text{Discard Carry } 100000011 \\ \text{Sum: } 00000011 \end{array}$$



8  
+ -5  
-----  
3  
www.vlsifacts.com

3 => 00000011

Step-3: The Final carry is 1 indicating that answer is positive and drop the carry

# ELECTRONIC PRINCIPLES AND DEVICES

## Binary Subtraction using 2's Complement

# Binary Subtraction using 2's Complement

4) Subtract  $(1010)_2$  from  $(1111)_2$  using 2's complement method.

Solution:  $(1111)_2 - (1010)_2 = ?$

Step-1: 2's complement of  $(1010)_2$  is  $(0110)_2$ .

Step-2: Add  $(0110)_2$  to  $(1111)_2$ . This is shown below.

$$\begin{array}{r}
 1\ 1\ 1\ 1 \\
 +\quad 0\ 1\ 1\ 0 \\
 \hline
 \text{Omit this carry } 1\ 0\ 1\ 0\ 1 \\
 \downarrow \\
 \boxed{0\ 1\ 0\ 1} \text{ Answer}
 \end{array}$$

Step-3: The Final carry is 1 indicating that answer is positive and drop the carry

# ELECTRONIC PRINCIPLES AND DEVICES

## Binary Subtraction using 2's Complement

5) Subtract  $(1010)_2$  from  $(1000)_2$  using 2's complement.

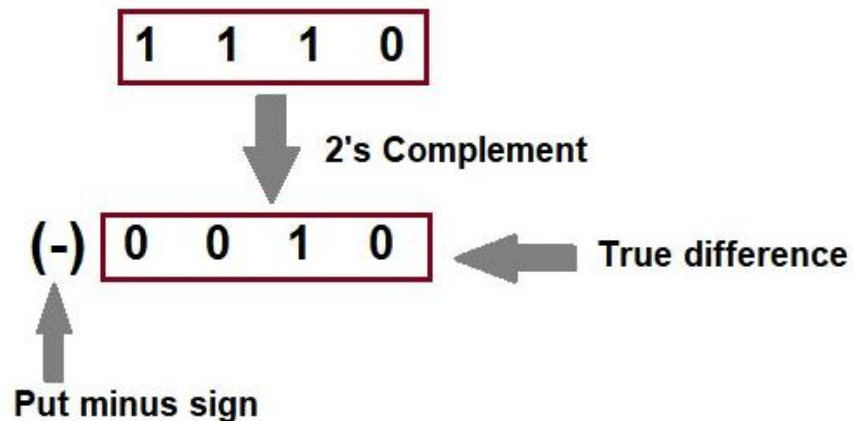
**Solution:**

Step-1: The 2's complement of  $(1010)_2$  is  $(0110)_2$ .

Step-2: Add  $(0110)_2$  to  $(1000)_2$ .

$$\begin{array}{r} 1\ 0\ 0\ 0 \\ +\ 0\ 1\ 1\ 0 \\ \hline 1\ 1\ 1\ 0 \end{array}$$

Step-3: No final carry, i.e, answer is negative. So, take 2's complement of the final sum and it is negative.



# ELECTRONIC PRINCIPLES AND DEVICES

## Binary Subtraction using 2's Complement

---

- 6) Given the two binary numbers  $X = 1010100$  and  $Y = 1000011$ , perform the subtraction (a)  $X - Y$  and (b)  $Y - X$  by using 2's complements.

$$\begin{array}{rcl} \text{(a)} & X = & 1010100 \\ & 2\text{'s complement of } Y = + & \underline{0111101} \\ & \text{Sum} = & 10010001 \\ & \text{Discard end carry } 2^7 = - & \underline{10000000} \\ & \text{Answer: } X - Y = & 0010001 \end{array}$$

$$\begin{array}{rcl} \text{(b)} & Y = & 1000011 \\ & 2\text{'s complement of } X = + & \underline{0101100} \\ & \text{Sum} = & 1101111 \end{array}$$

There is no end carry. Therefore, the answer is  $Y - X = -(2\text{'s complement of } 1101111) = -0010001$ .

# ELECTRONIC PRINCIPLES AND DEVICES

## Binary Subtraction using 2's Complement

---



7) Subtract 100011 from 10101 using 2's complement method

Step 1: Make equal number of bits

Minuend= 010101

Subtrahend=100011

Step 2: The 2's complement of  $(100011)_2$  is  $(011101)_2$ .

Step 3: Add  $(011101)_2$  with minuend  $(010101)_2$

$$\begin{array}{r} 011101 \\ + 010101 \\ \hline 110010 \end{array}$$

Step-3: No final carry, i.e, answer is negative. So, take 2's complement of the final sum and it is negative

**Final answer: -001110**

# ELECTRONIC PRINCIPLES AND DEVICES

## Binary Subtraction using 2's Complement



**-A-B=? (If both the numbers are negative)**

**Steps to find -A-B=?**

1. At first, find 2's complement of the A and B.
2. Add the 2's complements of A and B.
3. Discard the final carry.
4. Then 2's complement of the sum is the final result and it is negative.

Example: -5-3=?

Binary representation of 5 is 0101

Binary representation of 3 is 0011

2's complement of 5 is 1011

2's complement of 3 is 1101

Add the 2's complements of 5 and 3

$$\begin{array}{r} 1011 \quad (-5) \\ + 1101 \quad (-3) \\ \hline 1\ 1000 \quad (-8) \end{array}$$

Discard the carry, take 2's complement of the final sum and it is negative

Final answer: -1000

- ❖ Digital Electronic circuits process data that contains **binary values 1's and 0's**.
- ❖ A **Logic "1"** is also referred as HIGH voltage or TRUE or ON state
- ❖ A **Logic "0"** is also referred as LOW voltage or FALSE or OFF state
- ❖ A binary digit "0" or "1" is called a **bit**
- ❖ Digital information is stored using a series of binary values 1's and 0's.
- ❖ Digital systems such as digital telephones, digital cameras, computers, handheld portable devices and other high technology systems **stores and process** these binary values (1's and 0's)

## Boolean Algebra and Logic gates

---

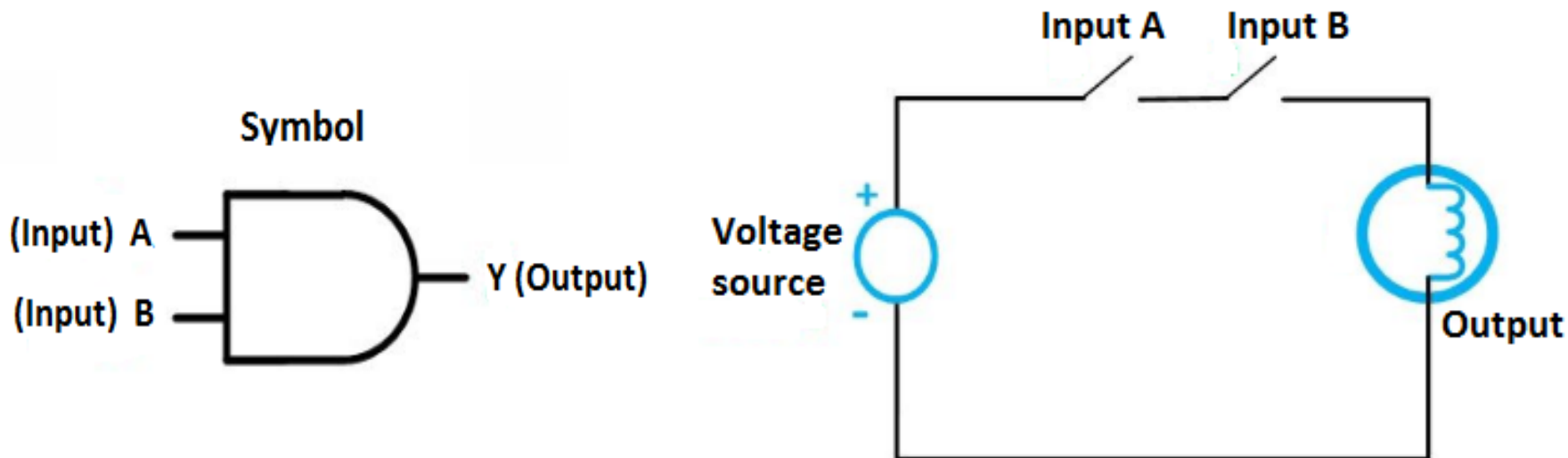
- ❖ Boolean Algebra is the mathematics used to analyse and **simplify logic or digital circuits**.
- ❖ Digital circuits are constructed by using **logic gates**.
- ❖ Logic gates are the **basic building blocks of digital systems, performs logical functions based on Boolean algebra**.
- ❖ Logic gates have **one or more inputs and only one output**.
- ❖ Number of possible input states is  **$2^n$** . Where n is Number of inputs
- ❖ Logic gates are implemented using diodes or transistors which acts as electronic switches
- ❖ logic gates : **(i) Basic Gates:** AND, OR, NOT,  
**(ii) Derived Gates:** NAND, NOR, EXOR and EXNOR gates

## Boolean Algebra and Logic gates

❖ **AND Gate** is an electronic circuit that gives output as logic “1” (HIGH) only if all inputs are “1”. Otherwise output is Logic “0” (LOW)

❖ Boolean algebra representation for AND gate is  $Y =$

$A \cdot B$   
❖ AND gate is represented as **series connection** of two switches

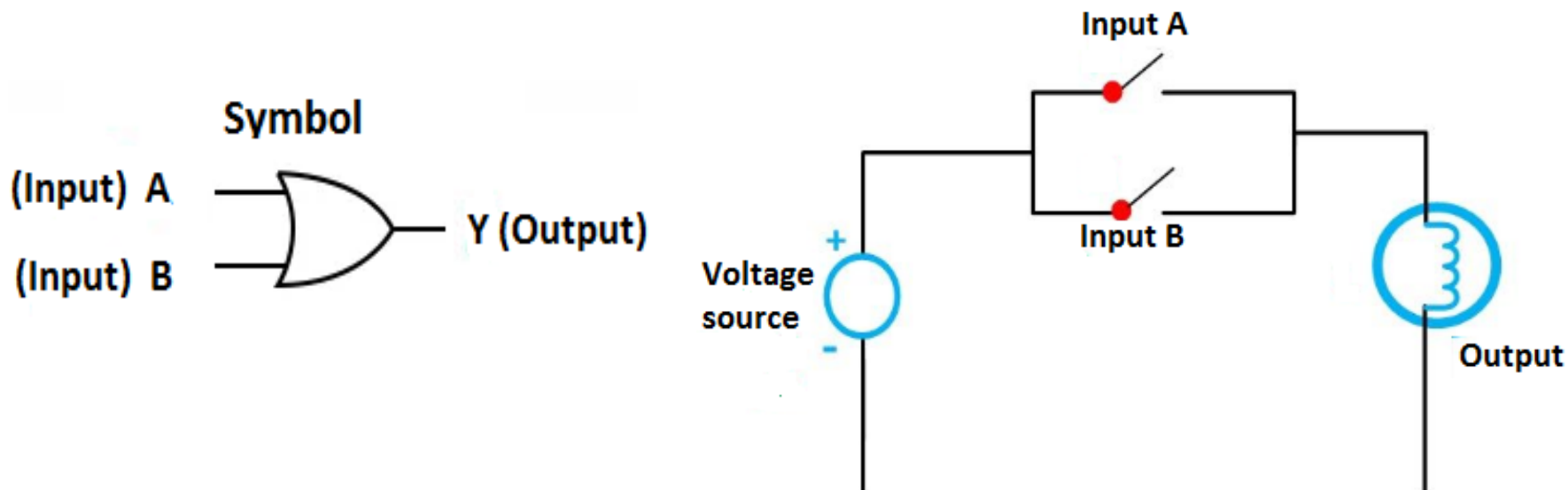


**Truth Table**

| Inputs |   | Output |
|--------|---|--------|
| A      | B | Y      |
| 0      | 0 | 0      |
| 0      | 1 | 0      |
| 1      | 0 | 0      |
| 1      | 1 | 1      |

## Boolean Algebra and Logic gates

- ❖ **OR Gate** is an electronic circuit that gives as Logic “1” (HIGH) if any of the inputs are HIGH .Output of OR gate goes Logic “0” (LOW) only if all inputs are Logic “0” (LOW)
- ❖ Boolean algebra representation of OR gate is  $Y = A + B$
- ❖ OR gate is represented as **parallel connection** of two switches



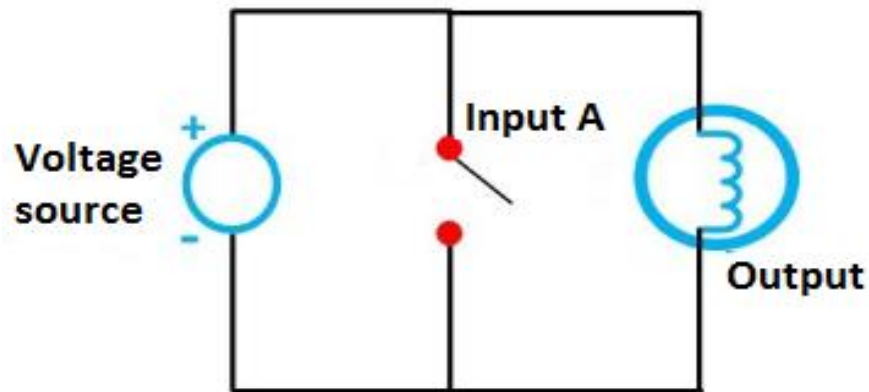
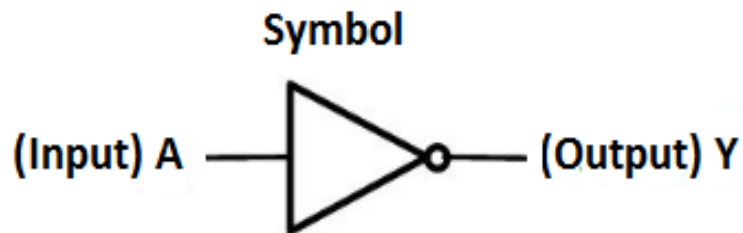
**Truth Table**

| Inputs |   | Output |
|--------|---|--------|
| A      | B | Y      |
| 0      | 0 | 0      |
| 0      | 1 | 1      |
| 1      | 0 | 1      |
| 1      | 1 | 1      |

❖ **NOT Gate** is a single input and single output gate which performs the inversion of the applied binary input signal. Hence it is also called as Inverter Gate.

❖ Boolean expression for NOT gate is  $Y = \overline{A}$

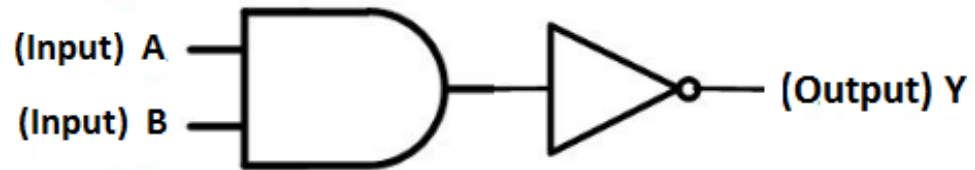
❖ NOT Gate can be represented by connecting a switch parallel to bulb



Truth Table

| Input | Output |
|-------|--------|
| A     | Y      |
| 0     | 1      |
| 1     | 0      |

### NAND Gate



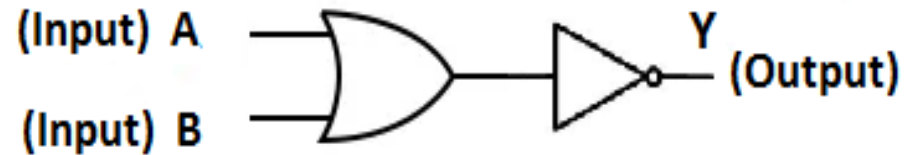
Symbol



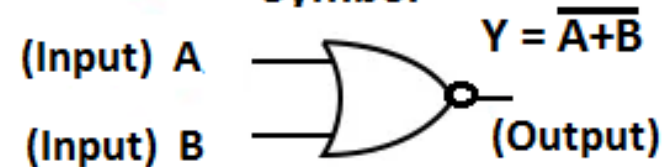
Truth Table

| NAND  |   |        |
|-------|---|--------|
| Input |   | Output |
| A     | B | Y      |
| 0     | 0 | 1      |
| 0     | 1 | 1      |
| 1     | 0 | 1      |
| 1     | 1 | 0      |

### NOR Gate



Symbol



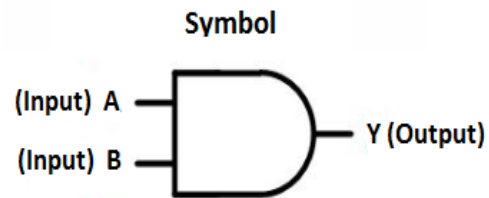
Truth Table

| NOR   |   |        |
|-------|---|--------|
| Input |   | Output |
| A     | B | Y      |
| 0     | 0 | 1      |
| 0     | 1 | 0      |
| 1     | 0 | 0      |
| 1     | 1 | 0      |

### ❖ Logical Boolean Laws:

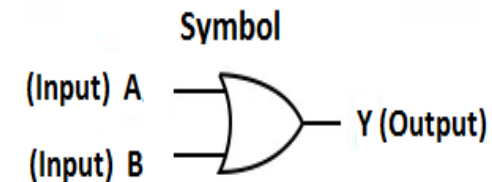
❖ **NOT / Inversion Law:**  $\overline{1} = 0$  (i) If  $A = 1$  then  $A' = 0$  and if  $A = 0$  then  $A' = 1$   
 $\overline{0} = 1$  (ii)  $(A')' = A$  itself

### ❖ **AND Law:**



|           |            |
|-----------|------------|
| $0.0 = 0$ | $A.0 = 0$  |
| $0.1 = 0$ | $A.1 = A$  |
| $1.0 = 0$ | $A.A = A$  |
| $1.1 = 1$ | $A.A' = 0$ |

### ❖ **OR Law:**



|           |            |
|-----------|------------|
| $0+0 = 0$ | $A+0 = A$  |
| $0+1 = 1$ | $A+1 = 1$  |
| $1+0 = 1$ | $A+A = A$  |
| $1+1 = 1$ | $A+A' = 1$ |

## Boolean Algebra and Logic gates

### ❖ Principle of duality in Boolean Algebra

❖ In Boolean Algebra , One type of expression can be converted into another type of expression by replacing “0 with 1 “ , “ 1 with 0” , “(+) sign with (.) sign “ and vice versa.

### Example:

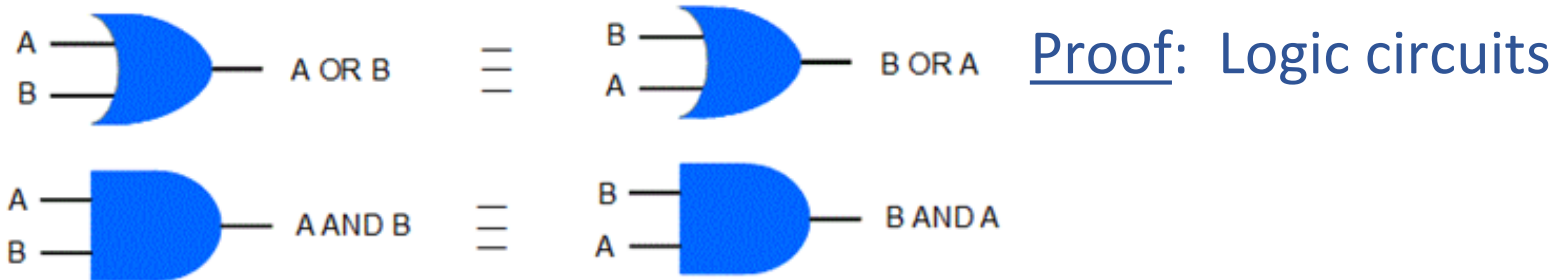
| AND (.)   | OR (+)    |
|-----------|-----------|
| $0.0 = 0$ | $1+1 = 1$ |
| $0.1 = 0$ | $1+0 = 1$ |
| $1.0 = 0$ | $0+1 = 1$ |
| $1.1 = 1$ | $0+0 = 0$ |

| Expression           | Dual Expression        |
|----------------------|------------------------|
| $\overline{0} = 1$   | $\overline{1} = 0$     |
| $0.1 = 0$            | $1 + 0 = 1$            |
| $A.0 = 0$            | $A + 1 = 1$            |
| $A.B = B.A$          | $A + B = B + A$        |
| $A.\overline{A} = 0$ | $A + \overline{A} = 1$ |

## Basic Theorem and Properties of Boolean Algebra

❖ Boolean Laws and Theorems are used to simplify the Boolean expressions.  
Hence reduce the number of logic gates.

❖ Commutative Laws: (i)  $A + B = B + A$   
(ii)  $A \cdot B = B \cdot A$



| A | B | (A+B) | (B+A) | (A.B) | (B.A) |
|---|---|-------|-------|-------|-------|
| 0 | 0 | 0     | 0     | 0     | 0     |
| 0 | 1 | 1     | 1     | 0     | 0     |
| 1 | 0 | 1     | 1     | 0     | 0     |
| 1 | 1 | 1     | 1     | 1     | 1     |

Proof: Truth  
Table

## Basic Theorems and Properties of Boolean Algebra

### ❖ Associative Law:

$$(A+B)+C = A+(B+C) \quad \text{Boolean addition}$$

### ❖ Proof:

| A | B | C | A + B | (A + B) + C | B + C | A + (B + C) |
|---|---|---|-------|-------------|-------|-------------|
| 0 | 0 | 0 | 0     | 0           | 0     | 0           |
| 0 | 0 | 1 | 0     | 1           | 1     | 1           |
| 0 | 1 | 0 | 1     | 1           | 1     | 1           |
| 0 | 1 | 1 | 1     | 1           | 1     | 1           |
| 1 | 0 | 0 | 1     | 1           | 0     | 1           |
| 1 | 0 | 1 | 1     | 1           | 1     | 1           |
| 1 | 1 | 0 | 1     | 1           | 1     | 1           |
| 1 | 1 | 1 | 1     | 1           | 1     | 1           |

### ❖ From principle of duality: $(A.B).C = A.(B.C)$ Boolean Multiplication

## Basic Theorem and Properties of Boolean Algebra

### ❖ Distributive Law:

$$A + B.C = (A+B) . (A+C)$$

### ❖ Proof: Truth

Table

| A | B | C | BC | A+BC | (A+B) | (A+C) | (A+B)(A+C) |
|---|---|---|----|------|-------|-------|------------|
| 0 | 0 | 0 | 0  | 0    | 0     | 0     | 0          |
| 0 | 0 | 1 | 0  | 0    | 0     | 1     | 0          |
| 0 | 1 | 0 | 0  | 0    | 1     | 0     | 0          |
| 0 | 1 | 1 | 1  | 1    | 1     | 1     | 1          |
| 1 | 0 | 0 | 0  | 1    | 1     | 1     | 1          |
| 1 | 0 | 1 | 0  | 1    | 1     | 1     | 1          |
| 1 | 1 | 0 | 0  | 1    | 1     | 1     | 1          |
| 1 | 1 | 1 | 1  | 1    | 1     | 1     | 1          |

### Dual of distributive law:

$$A . (B+C) = A.B + A.C$$

### ❖ Proof: Truth

Table

| A | B | C | B+C | A(B+C) | AB | AC | AB+AC |
|---|---|---|-----|--------|----|----|-------|
| 0 | 0 | 0 | 0   | 0      | 0  | 0  | 0     |
| 0 | 0 | 1 | 1   | 0      | 0  | 0  | 0     |
| 0 | 1 | 0 | 1   | 0      | 0  | 0  | 0     |
| 0 | 1 | 1 | 1   | 0      | 0  | 0  | 0     |
| 1 | 0 | 0 | 0   | 0      | 0  | 0  | 0     |
| 1 | 0 | 1 | 1   | 1      | 0  | 1  | 1     |
| 1 | 1 | 0 | 1   | 1      | 1  | 0  | 1     |
| 1 | 1 | 1 | 1   | 1      | 1  | 1  | 1     |

## Basic Theorem and Properties of Boolean Algebra

### ❖ De Morgan's Theorem :

(i) The complement of the sum of 2 variables is equal to the product of the complements of individual variables:

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

(ii) The complement of the product of 2 variables is equal to the sum of the complements of individual variables:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

| A | B | $\overline{A}$ | $\overline{B}$ | A+B | A.B | $\overline{A+B}$ | $\overline{A} \cdot \overline{B}$ | $\overline{A \cdot B}$ | $\overline{A} + \overline{B}$ |
|---|---|----------------|----------------|-----|-----|------------------|-----------------------------------|------------------------|-------------------------------|
| 0 | 0 | 1              | 1              | 0   | 0   | 1                | 1                                 | 1                      | 1                             |
| 0 | 1 | 1              | 0              | 1   | 0   | 0                | 0                                 | 1                      | 1                             |
| 1 | 0 | 0              | 1              | 1   | 0   | 0                | 0                                 | 1                      | 1                             |
| 1 | 1 | 0              | 0              | 1   | 1   | 0                | 0                                 | 0                      | 0                             |

## Basic Theorem and Properties of Boolean Algebra

### ❖ Absorption Theorem:

#### (i) $A+AB = A$

$$\begin{aligned}\text{LHS: } &= A + AB \\ &= A.1 + AB \rightarrow \text{since } A.1 = A \\ &= A(1+B) \rightarrow \text{since } 1+B = 1 \\ &= A.1 \\ &= A = \text{RHS}\end{aligned}$$

#### (ii) $A(A+B) = A$

$$\begin{aligned}\text{LHS} &= A(A+B) \\ &= A.A + A.B \\ &= A+AB \rightarrow \text{since } A.A = A \\ &= A(1+B) \\ &= A.1 \\ &= A = \text{RHS}\end{aligned}$$

#### (iii) $A+\bar{A}B = A+B$

$$\begin{aligned}\text{LHS} &= A + \bar{A}B \\ &= (A + \bar{A})(A + B) \rightarrow \text{since } A+BC = (A+B)(A+C) \\ &= (1).(A+B) \rightarrow \text{since } A + \bar{A} = 1 \\ &= A + B = \text{RHS}\end{aligned}$$

#### (iv) $A.(\bar{A}+B) = AB$

$$\begin{aligned}\text{LHS} &= A.(\bar{A} + B) \\ &= A.\bar{A} + A.B \rightarrow (A.\bar{A} = 0) \\ &= AB = \text{RHS}\end{aligned}$$

❖ Redundancy Laws

## Basic Theorem and Properties of Boolean Algebra

### ❖ Consensus Theorem:

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

$$\begin{aligned}\text{LHS} &= AB + \bar{A}C + BC \\ &= AB + \bar{A}C + BC.1 \\ &= AB + \bar{A}C + BC (A + \bar{A}) \rightarrow \text{since } A + \bar{A} = 1 \\ &= AB + \bar{A}C + ABC + \bar{A}BC \\ &= AB (1 + C) + \bar{A}C (1 + B) \\ 1 + B &= 1 + C = 1 \\ &= AB + \bar{A}C = \text{RHS}\end{aligned}$$

❖ **BC is redundant term**

### ❖ Dual of consensus theorem:

$$\begin{aligned}(A+B)(\bar{A}+C)(B+C) &= (A+B)(\bar{A}+C) \\ \text{LHS} &= (A+B)(\bar{A}+C)(B+C) \\ \text{WKT } A+BC &= (A+B)(A+C) \\ B+AC &= (B+A)(B+C) \\ \text{LHS} &= (B+AC)(\bar{A}+C) \\ &= \bar{A}B + \bar{A}AC + BC + ACC \\ &= \bar{A}B + BC + AC + A\bar{A} \\ &= B(\bar{A} + C) + A(\bar{A} + C) \\ &= (A+B)(\bar{A}+C) = \text{RHS}\end{aligned}$$

## Boolean Functions, Canonical and Standard Form

---

- ❖ Boolean function described by an algebraic expression consists of **binary variables, the constants 0 and 1, and the logic operation symbols**.
- ❖ Boolean function is **evaluated to logic-1 or logic-0** for a given value of the binary variables.
- ❖ Boolean function can be represented in a **truth table**.
- ❖ A Boolean function can be implemented as **digital circuit**, Which is constructed by using logic gates.
- ❖ Example:  $F = X + Y'Z$

$F = 1$     if  $X = 1$  or if  $Y = 0$  and  $Z = 1$

$F = 0$     Otherwise

## Boolean Functions, Canonical and Standard Form

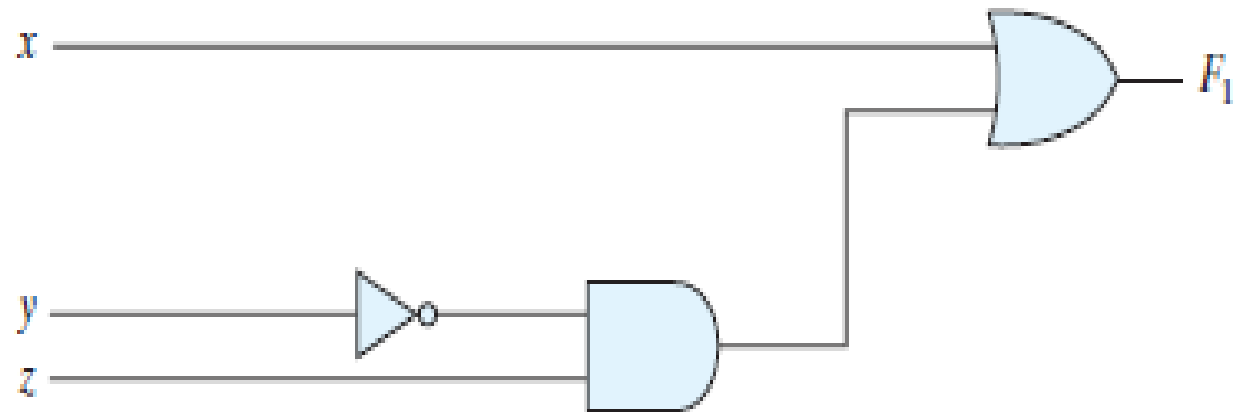
❖ Example:  $F_1 = x + y'z$

❖  $F_1$  contains either 0 or 1 for each of these combinations. The table shows that the function is equal to 1 when  $x = 1$  or when  $yz = 01$  and is equal to 0 otherwise.

**Truth Table:**

| $x$ | $y$ | $z$ | $F_1$ |
|-----|-----|-----|-------|
| 0   | 0   | 0   | 0     |
| 0   | 0   | 1   | 1     |
| 0   | 1   | 0   | 0     |
| 0   | 1   | 1   | 0     |
| 1   | 0   | 0   | 1     |
| 1   | 0   | 1   | 1     |
| 1   | 1   | 0   | 1     |
| 1   | 1   | 1   | 1     |

**Gate Level Implementation:**



❖ Simplify and realize the given Boolean function using basic gates

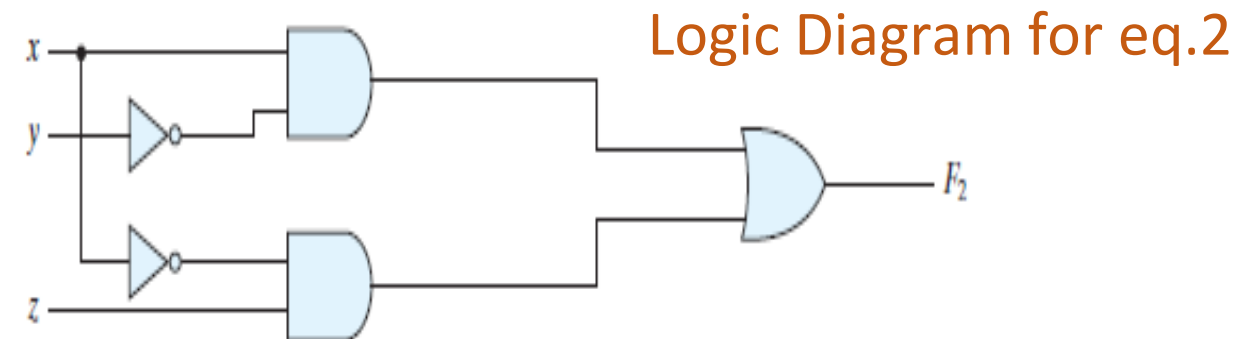
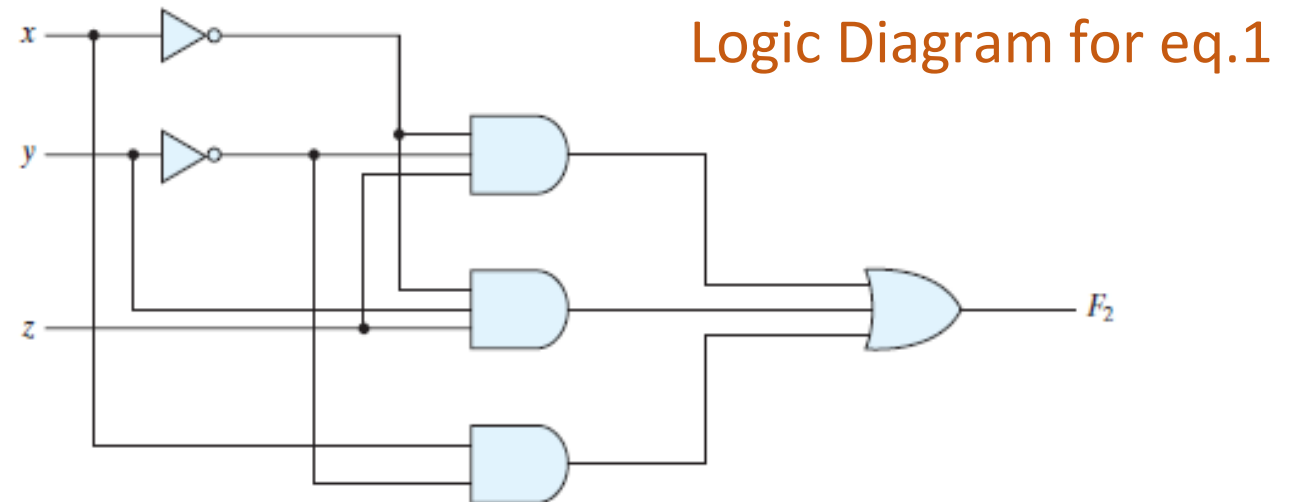
$$F_2 = x'y'z + x'yz + xy' \dots \dots \dots (1)$$

$$F_2 = x'z (y' + y) + xy'$$

$$F_2 = x'z + xy' \dots \dots \dots (2)$$

Truth Table

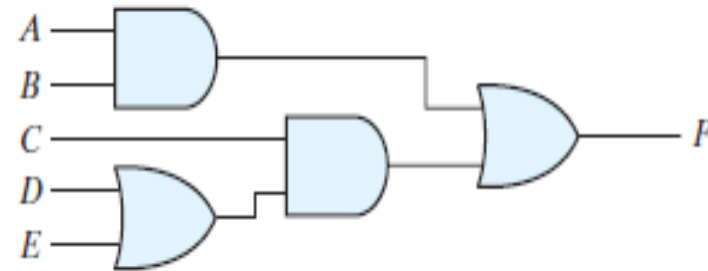
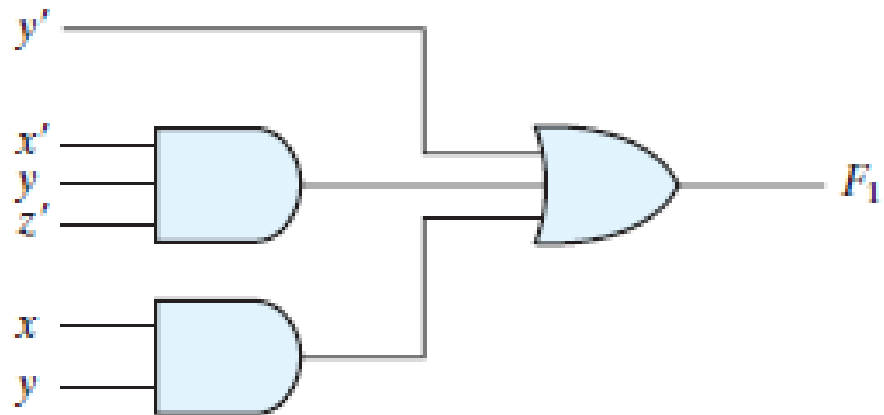
| $x$ | $y$ | $z$ | $F_2$ |
|-----|-----|-----|-------|
| 0   | 0   | 0   | 0     |
| 0   | 0   | 1   | 1     |
| 0   | 1   | 0   | 0     |
| 0   | 1   | 1   | 1     |
| 1   | 0   | 0   | 1     |
| 1   | 0   | 1   | 1     |
| 1   | 1   | 0   | 0     |
| 1   | 1   | 1   | 0     |



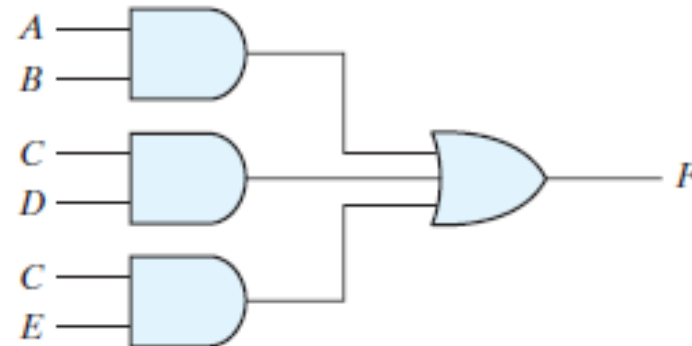
### ❖ Sum of Products: (SOP)

- The sum of products is a Boolean expression containing **AND** terms, called **product terms**, with one or more literals each. The **sum** denotes the **OR**ing of these terms.
- Two- Level Logic implementation of SOP

#### ❖ Example 1 : $F_1 = y' + xy + x'yz'$



Example 2:  
 $F = AB + C(D+E)$



$F = AB + CD + CE$

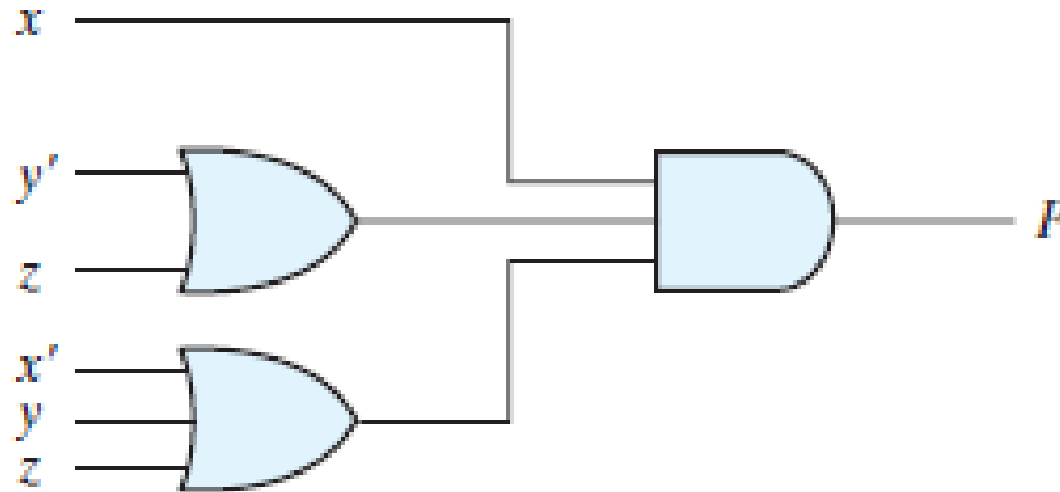
## Boolean Functions, Canonical and Standard Form

### ❖ Product Of Sum: POS

➤ A product of sum (POS) is a Boolean expression containing OR terms, called sum terms. Each term may have any number of literals. The product denotes the ANDing of these terms

➤ Example 1 :

$$F = x.(y' + z).(x' + y + z)$$



➤ Example 2:  $Y = (A+B). (C+D)$

### ❖ Canonical SOP Form:

➤ Each product term contains all the literals of that function either in true or complement form

➤ **Example:**  $F(x,y,z) = xyz + x'y'z + x'yz' + x'y'z'$

➤ Each product term is called as **minterm**

➤ For three variable function: Truth Table

| <i>x</i> | <i>y</i> | <i>z</i> | Minterms |             |
|----------|----------|----------|----------|-------------|
|          |          |          | Term     | Designation |
| 0        | 0        | 0        | $x'y'z'$ | $m_0$       |
| 0        | 0        | 1        | $x'y'z$  | $m_1$       |
| 0        | 1        | 0        | $x'yz'$  | $m_2$       |
| 0        | 1        | 1        | $x'yz$   | $m_3$       |
| 1        | 0        | 0        | $xy'z'$  | $m_4$       |
| 1        | 0        | 1        | $xy'z$   | $m_5$       |
| 1        | 1        | 0        | $xyz'$   | $m_6$       |
| 1        | 1        | 1        | $xyz$    | $m_7$       |

❖ Consider the function:  $f_1$  and  $f_2$  in the truth table

| $x$ | $y$ | $z$ | Function $f_1$ | Function $f_2$ |
|-----|-----|-----|----------------|----------------|
| 0   | 0   | 0   | 0              | 0              |
| 0   | 0   | 1   | 1              | 0              |
| 0   | 1   | 0   | 0              | 0              |
| 0   | 1   | 1   | 0              | 1              |
| 1   | 0   | 0   | 1              | 0              |
| 1   | 0   | 1   | 0              | 1              |
| 1   | 1   | 0   | 0              | 1              |
| 1   | 1   | 1   | 1              | 1              |

❖ Canonical SOP Form for  $f_1$

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

❖ Canonical SOP Form for  $f_2$

$$f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

### ❖ Canonical POS Form:

➤ Each sum term contains all the literals of that function either in true or complement form

➤ **Example:**  $F(x, y, z) = (x + y + z) (x' + y' + z) (x' + y + z')$

➤ Each sum term is called as **Maxterm**

➤ For three variable function: Truth Table

| <i>x</i> | <i>y</i> | <i>z</i> | Maxterms       |             |
|----------|----------|----------|----------------|-------------|
|          |          |          | Term           | Designation |
| 0        | 0        | 0        | $x + y + z$    | $M_0$       |
| 0        | 0        | 1        | $x + y + z'$   | $M_1$       |
| 0        | 1        | 0        | $x + y' + z$   | $M_2$       |
| 0        | 1        | 1        | $x + y' + z'$  | $M_3$       |
| 1        | 0        | 0        | $x' + y + z$   | $M_4$       |
| 1        | 0        | 1        | $x' + y + z'$  | $M_5$       |
| 1        | 1        | 0        | $x' + y' + z$  | $M_6$       |
| 1        | 1        | 1        | $x' + y' + z'$ | $M_7$       |

❖ Consider the function: f1 and f2 in the truth table

| x | y | z | Function $f_1$ | Function $f_2$ |
|---|---|---|----------------|----------------|
| 0 | 0 | 0 | 0              | 0              |
| 0 | 0 | 1 | 1              | 0              |
| 0 | 1 | 0 | 0              | 0              |
| 0 | 1 | 1 | 0              | 1              |
| 1 | 0 | 0 | 1              | 0              |
| 1 | 0 | 1 | 0              | 1              |
| 1 | 1 | 0 | 0              | 1              |
| 1 | 1 | 1 | 1              | 1              |

❖ Canonical POS Form for f1

$$\begin{aligned}f_1 &= (x + y + z)(x + y' + z)(x + y' + z')(x' + y + z') \\ &\quad (x' + y' + z) \\ &= M_0. M_2. M_3. M_5. M_6\end{aligned}$$

❖ Canonical POS Form for f2

$$\begin{aligned}f_2 &= (x + y + z)(x + y + z')(x + y' + z)(x' + y + z) \\ &= M_0. M_1. M_2. M_4\end{aligned}$$

❖ Find the Minterms for the given expression: (Convert into canonical SOP Form)

$$F = A + BC$$

$$F = A.1.1 + BC.1$$

$$F = A. (B+B'). (C+C') + BC.(A+A')$$

$$F = A. (BC + BC' + B'C + B'C') + ABC + A'BC$$

$$F = ABC + ABC' + AB'C + AB'C' + ABC + A'BC$$

$$F = ABC + ABC' + AB'C + AB'C' + A'BC$$

$$F = m_7 + m_6 + m_5 + m_4 + m_3$$

$$F = \Sigma (3,4,5,6,7)$$

❖ Convert the given expression  $F(A, B, C) = A + B'C$  into canonical SOP form

$$F = A + B'C$$

$$= A(B+B')(C+C') + B'C(A+A')$$

$$= (AB + AB')(C+C') + AB'C + A'B'C$$

$$= ABC + ABC' + AB'C + AB'C' + AB'C + A'B'C$$

$$= ABC + ABC' + AB'C + AB'C' + A'B'C$$

$$= m_7 + m_6 + m_5 + m_4 + m_1$$

$$F = \sum(1, 4, 5, 6, 7)$$

**Note:**

An alternative procedure for deriving the minterms of a Boolean function is to obtain the truth table of the function directly from the algebraic expression and then read the minterms from the truth table.

## Boolean Functions, Canonical and Standard Form

❖ Express the Boolean function  $F = x y + x' z$  as a product of maxterms.  
(Convert the given expression  $F = x y + x' z$  into canonical POS form).

First, convert the function into OR terms by using the distributive law,  $x + y z = (x + y)(x + z)$

$$\begin{aligned} F &= (x y + x') (x y + z) \\ &= (x + x') (y + x') (x + z) (y + z) \\ &= (x' + y) (x + z) (y + z) \\ &= (x' + y + z z') (x + z + y y') (y + z + x x') \\ &= (x' + y + z) (x' + y + z') (x + y + z) (x + y' + z) (x + y + z) (x' + y + z) \end{aligned}$$

$$= M_0 M_2 M_4 M_5$$

$$F(x, y, z) = \Pi(0, 2, 4, 5)$$

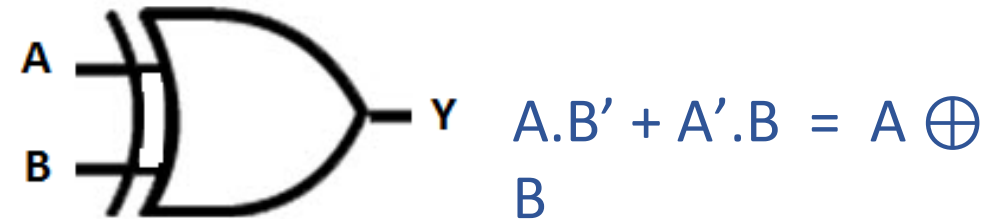
## Other Logical Operation, Digital Logic gates (XOR and XNOR)

- ❖ **XOR Gate** is a digital logic Gate which has two or more inputs and only one output that performs Exclusive OR operation. Hence it is also called as Ex-OR or XOR
- ❖ For two input XOR gate output is **logic-1** only when one of its input is logic-1 (unequal input i.e..A = 0 and B = 1 or A = 1 and B = 0).
- ❖ Output of XOR gate is **logic-0** if both inputs are same (i.e., A = 0 and B = 0 or A = 1 and B = 1)
- ❖ Truth Table of XOR

Gate:

| Input A | Input B | Output Y |
|---------|---------|----------|
| 0       | 0       | 0        |
| 0       | 1       | 1        |
| 1       | 0       | 1        |
| 1       | 1       | 0        |

❖ Symbol



## Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ XOR gate performs modulo sum operation without including carry.

i.e.,  $0+0 = 0$ ,  $0+1 = 1$ ,  $1+0 = 1$ ,  $1+1 = 0$  (carry 1)

❖ Truth Table

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

$$Y = A'.B + A.B'$$

$$Y = A \text{ XOR } B$$

$$Y = A \oplus B$$

$$A.B' + A'.B = A \oplus B$$

$$\text{LHS} = A.B' + A'.B$$

**Case1:** If  $A = 0$  and  $B = 0$

$$0.0' + 0'.0 = 0.1 + 1.0 = 0$$

**Case2:** If  $A = 0$  and  $B = 1$

$$0.1' + 0'.1 = 0.0 + 1.1 = 1$$

**Case3:** If  $A = 1$  and  $B = 0$

$$1.0' + 1'.0 = 1.1 + 0.0 = 1$$

**Case4:** If  $A = 1$  and  $B = 1$

$$1.1' + 1'.1 = 1.0 + 0.1 = 0$$

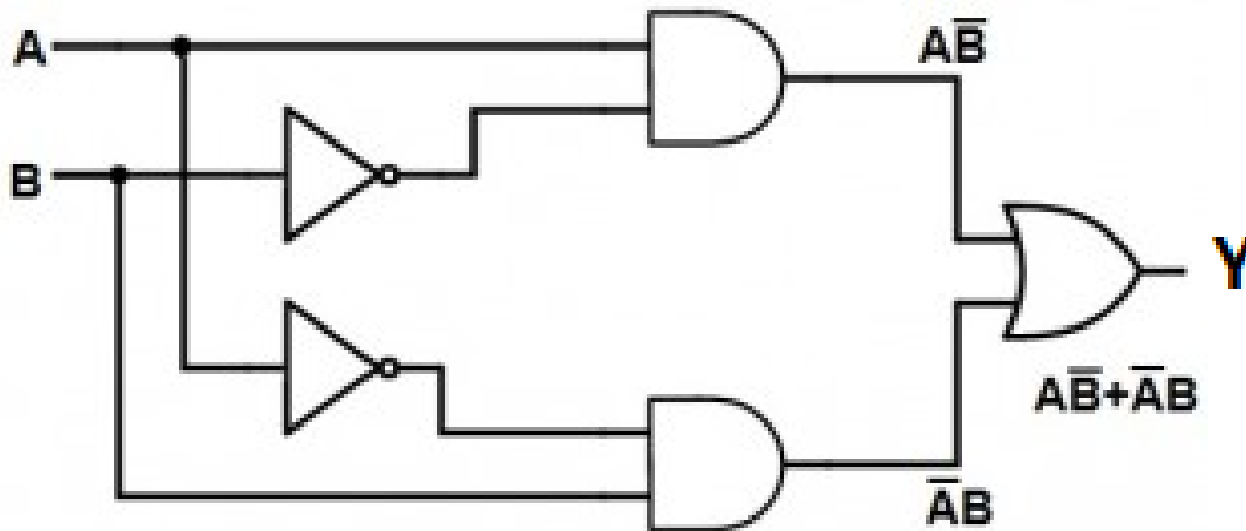
$$\text{Hence } Y = A \oplus B = A.B' + A'.B$$

## Other Logical Operation, Digital Logic gates (XOR and XNOR)

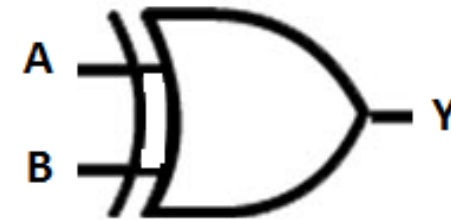
❖ Implementation of XOR Gate using basic gates:

$$Y = A \oplus B = A.B' + A'.B$$

❖ Logic Diagram



❖ Symbol



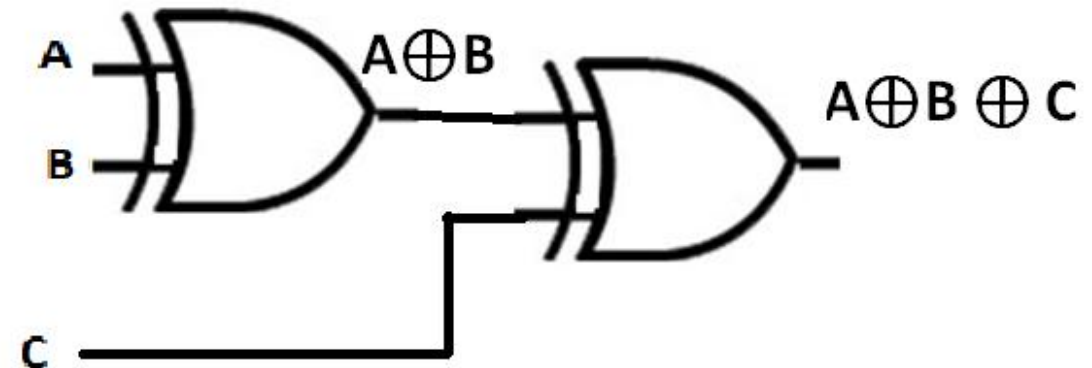
## Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ 3 - Input XOR Gate:  $Y = A \oplus B \oplus C$

Truth Table

| A | B | C | $A \oplus B$ | $A \oplus B \oplus C$ |
|---|---|---|--------------|-----------------------|
| 0 | 0 | 0 | 0            | 0                     |
| 0 | 0 | 1 | 0            | 1                     |
| 0 | 1 | 0 | 1            | 1                     |
| 0 | 1 | 1 | 1            | 0                     |
| 1 | 0 | 0 | 1            | 1                     |
| 1 | 0 | 1 | 1            | 0                     |
| 1 | 1 | 0 | 0            | 0                     |
| 1 | 1 | 1 | 0            | 1                     |

Symbol



Associative Law:

$$(A \oplus B) \oplus C = A \oplus (B \oplus C)$$

❖ Even or odd Parity Bits

## Other Logical Operation, Digital Logic gates (XOR and XNOR)

- ❖ **XNOR Gate:** Logical Complement of XOR Gate
- ❖ For two input XNOR Gate if both inputs are same i.e.,  $A = 0$  and  $B = 0$  or  $A = 1$  and  $B = 1$ . Then output of logic gate is logic-1 (High). Output of XNOR is logic-0 if inputs are unequal.
- ❖ Equality detector

**Truth Table**

| Input A | Input B | Output Y |
|---------|---------|----------|
| 0       | 0       | 1        |
| 0       | 1       | 0        |
| 1       | 0       | 0        |
| 1       | 1       | 1        |

❖ **Boolean Expression**

$$Y = A.B + A'.B'$$

$$Y = A \odot B$$

❖ **Symbol**



## Other Logical Operation, Digital Logic gates (XOR and XNOR)

### ❖ XNOR Gate:

$$Y = (A \oplus B)'$$

$$Y = (A.B' + A'.B)'$$

De Morgan's Theorem

$$Y = (A.B')' . (A'.B)'$$

$$Y = (A' + B) . (A + B')$$

$$Y = A'.A + A'.B' + A.B + B.B'$$

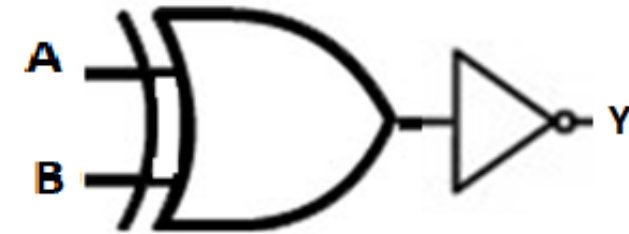
$$Y = 0 + A'.B' + A.B + 0$$

$$Y = A'.B' + A.B$$

$$Y = (A \oplus B)' = A \odot B$$

$$Y = (A \text{ XOR } B)' = (A \text{ XNOR } B)$$

### ❖ Logic Diagram:



### ❖ Symbol:

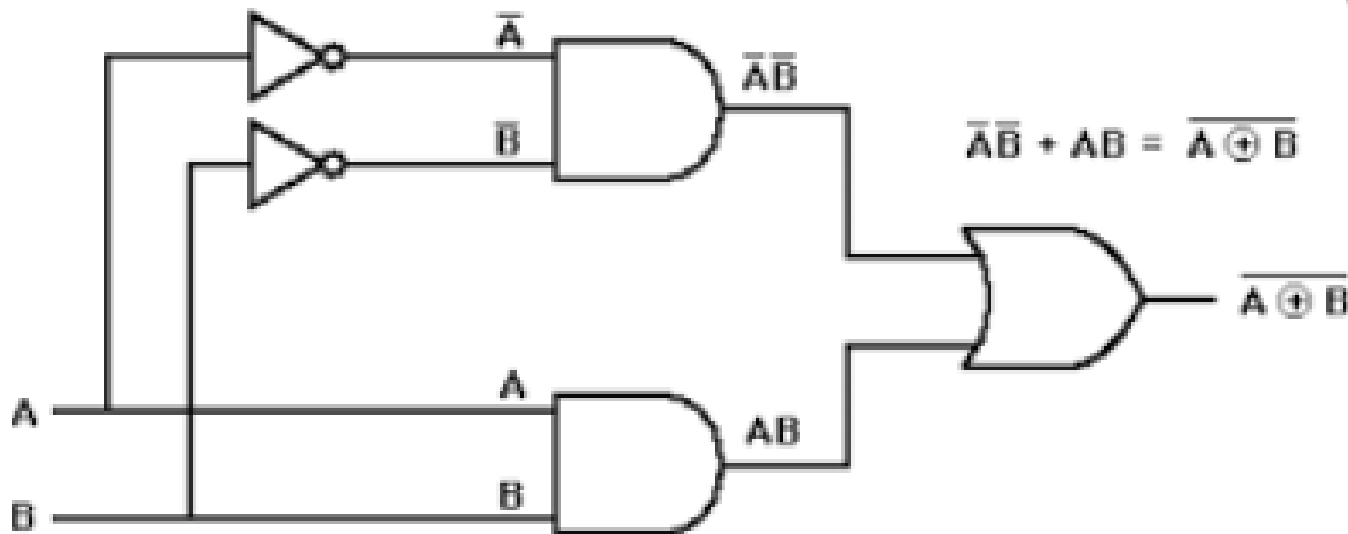


## Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ Implement the XNOR Gate using basic gates:

$$Y = A'.B' + A.B$$

❖ Logic Diagram:



$$Y = (A \oplus B)' = A \odot B$$

## Other Logical Operation, Digital Logic gates (XOR and XNOR)

### ❖ Properties of XOR Gate:

➤ Identity element:  $A \oplus 0 = A$

➤  $A \oplus 1 = A'$

➤  $A \oplus A = 0$

➤ Commutative Law:  $A \oplus B = B \oplus A$

➤ Associative Law :  $A \oplus (B \oplus C) = (A \oplus B) \oplus C$



| Input A | Input B | Output Y |
|---------|---------|----------|
| 0       | 0       | 0        |
| 0       | 1       | 1        |
| 1       | 0       | 1        |
| 1       | 1       | 0        |

## Other Logical Operation, Digital Logic gates (XOR and XNOR)

---

### ❖ Applications of XOR and XNOR

- XOR gate is used in processor's Arithmetic Logic Unit (ALU) for binary addition.
- XOR logic gate is used to generate pseudorandom numbers in hardware.
- To generate parity bits and error detection
- Equality detector

## Realization of Boolean expression using Universal Gates

---

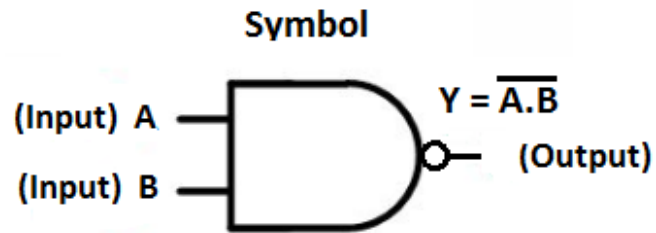
❖ **Universal Gates :** (i) NAND Gate  
(ii) NOR Gate

❖ Any digital logic circuit can be implemented by using NAND or NOR logic gates.

❖ NAND and NOR gates are easier to fabricate with electronic components and are used in all Integrated Circuit (IC's) digital logic families.

## Realization of Boolean expression using Universal Gates

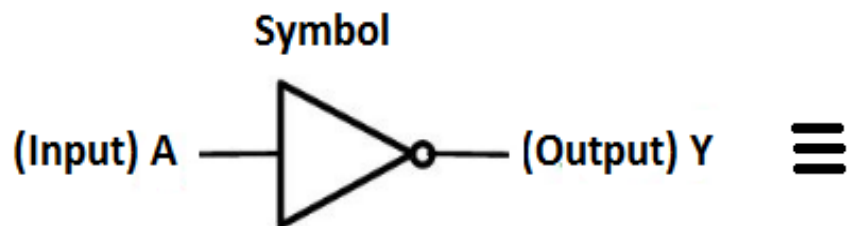
### ❖ Realization of logic gates using NAND Gates:



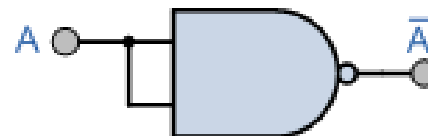
Truth Table

| NAND  |   |        |
|-------|---|--------|
| Input |   | Output |
| A     | B | Y      |
| 0     | 0 | 1      |
| 0     | 1 | 1      |
| 1     | 0 | 1      |
| 1     | 1 | 0      |

### ❖ NOT Gate:



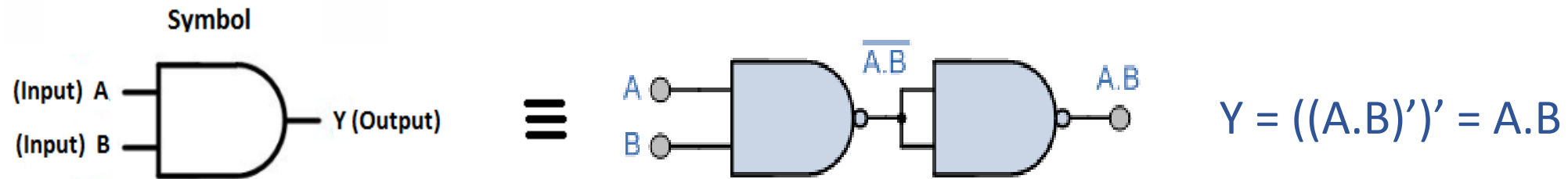
≡



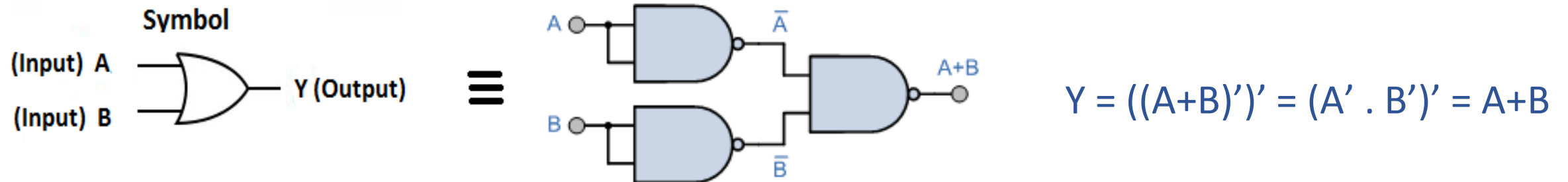
$$Y = (A.A)' = (A)' = A'$$

## Realization of Boolean expression using Universal Gates

### ❖ AND Gate:

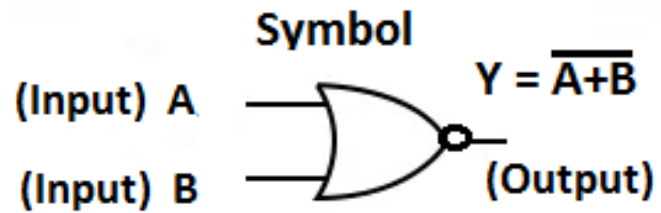


### ❖ OR Gate:

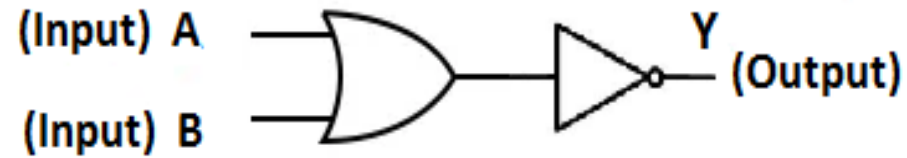


## Realization of Boolean expression using Universal Gates

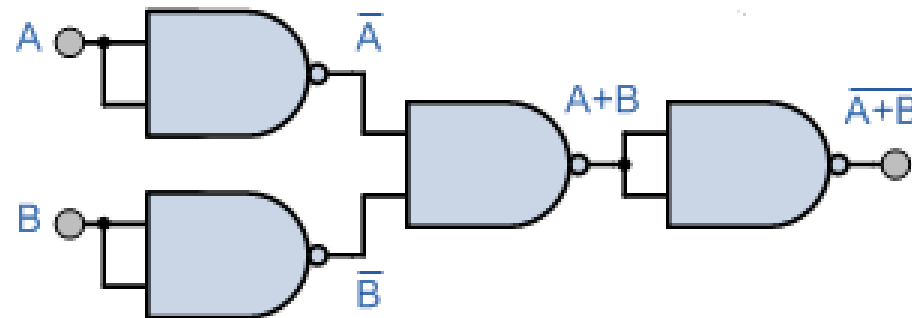
### ❖ NOR Gate:



≡

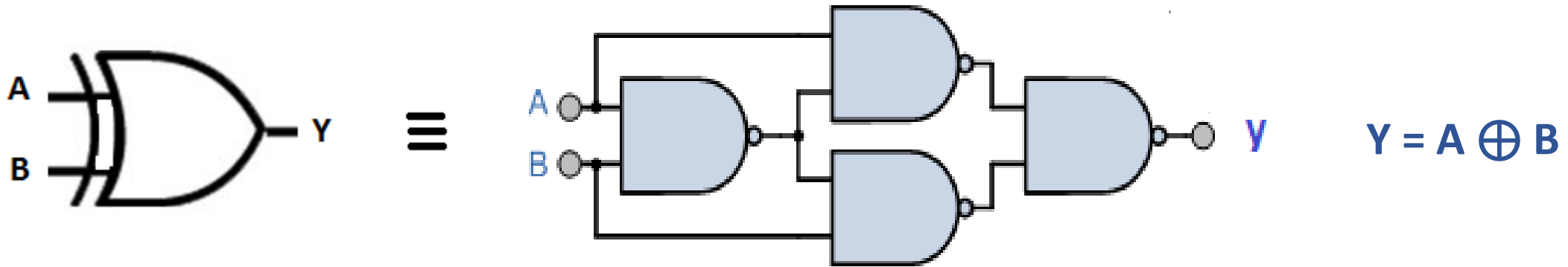


≡



## Realization of Boolean expression using Universal Gates

### ❖ XOR Gate:



$$Y = A.B' + A'B = A (A.B)' + B. (A.B)'$$

$$Y = ((A (A.B)' + B. (A.B)'))'$$

$$Y = (((A. (A.B)'))' . (B.(A.B)'))' )''$$

$$Y = A (A.B)' + B. (A.B)'$$

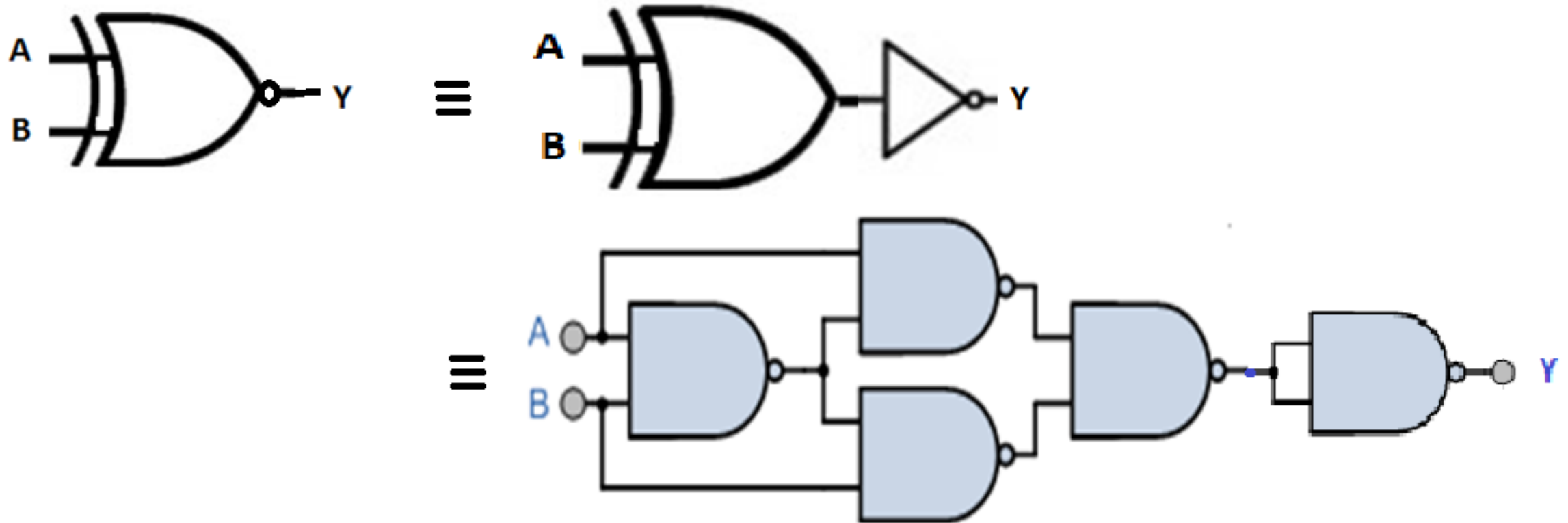
$$Y = A (A' + B') + B. (A' + B')$$

$$Y = A.A' + A.B' + A'.B + B.B'$$

$$Y = A.B' + A'.B$$

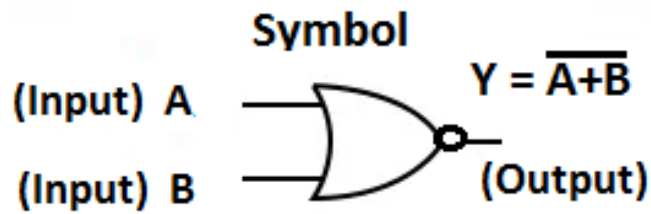
## Realization of Boolean expression using Universal Gates

### ❖ XNOR Gate:



## Realization of Boolean expression using Universal Gates

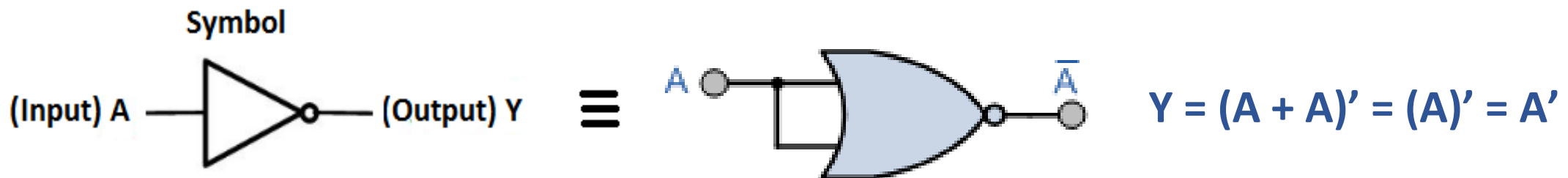
### ❖ Realization of logic gates using NOR Gates:



Truth Table

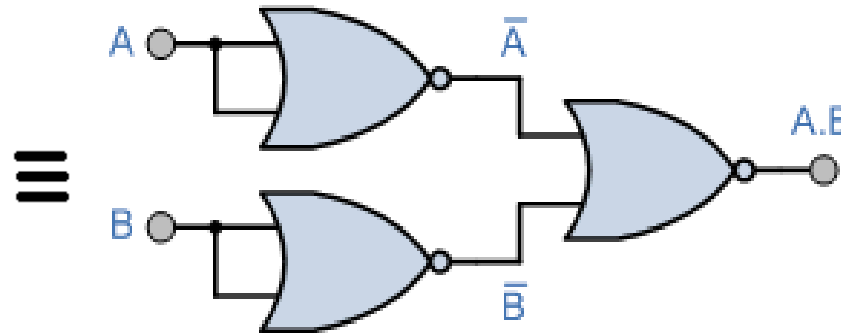
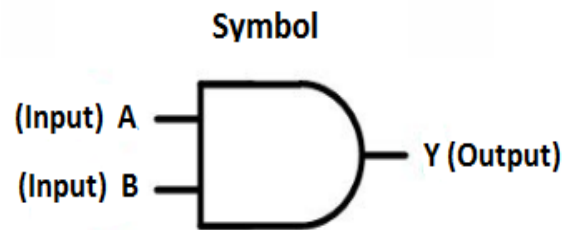
| NOR   |   |        |
|-------|---|--------|
| Input |   | Output |
| A     | B | Y      |
| 0     | 0 | 1      |
| 0     | 1 | 0      |
| 1     | 0 | 0      |
| 1     | 1 | 0      |

### ❖ NOT Gate:



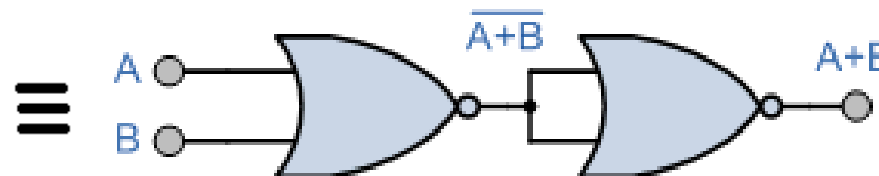
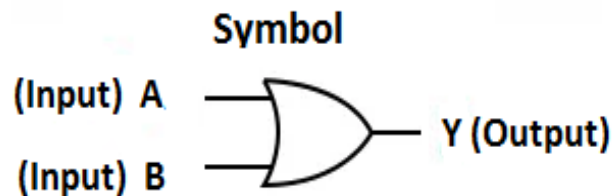
## Realization of Boolean expression using Universal Gates

### ❖ AND Gate:



$$Y = ((A.B)')' = (A' + B')' = A.B$$

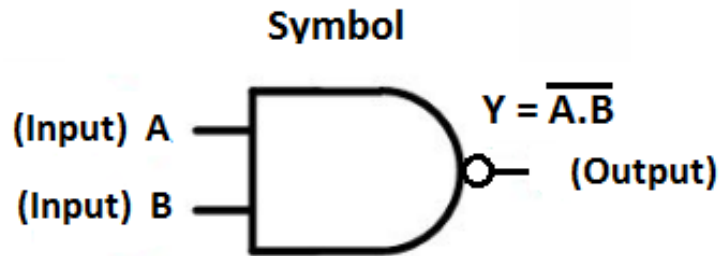
### ❖ OR Gate:



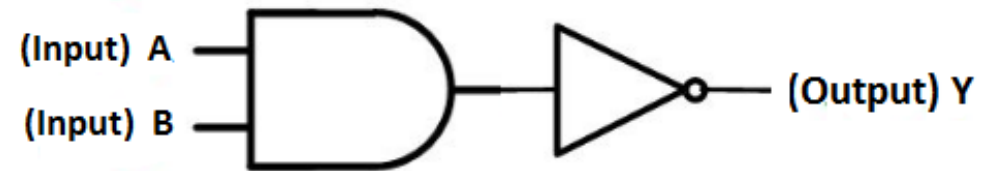
$$Y = ((A+B)')' = A + B$$

## Realization of Boolean expression using Universal Gates

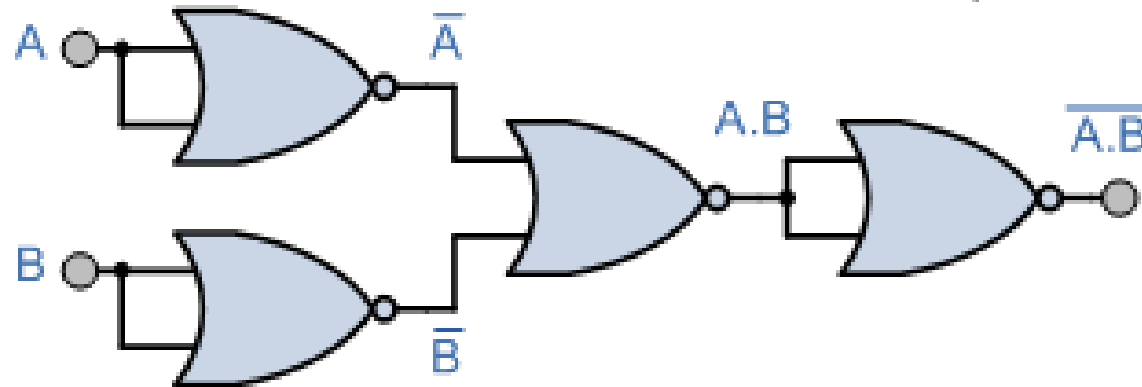
### ❖ NAND Gate:



≡

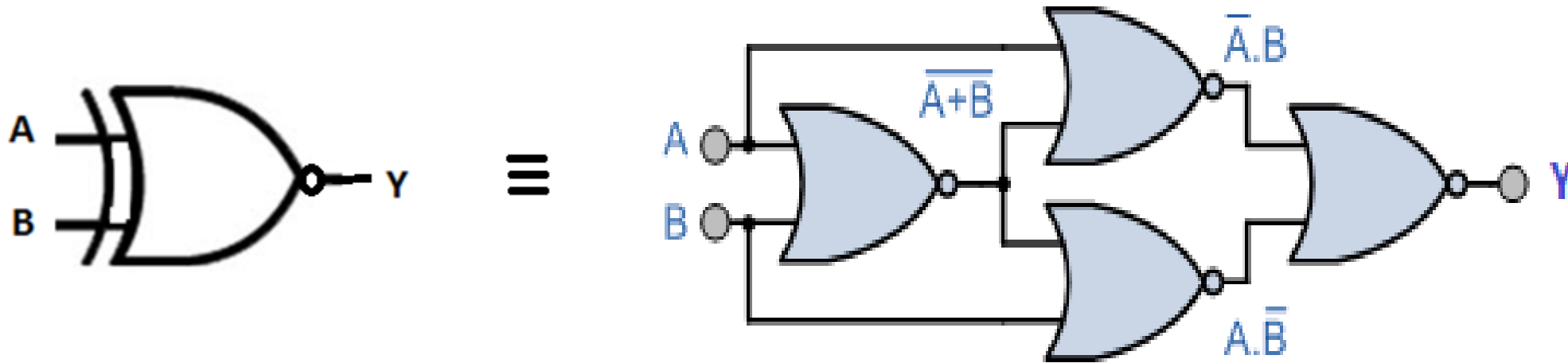


≡



## Realization of Boolean expression using Universal Gates

### ❖ XNOR Gate:



$$Y1 = ((A+B)' + A)' = (A+B). A' = A'.B$$

$$Y2 = ((A+B)' + B)' = (A+B). B' = A.B'$$

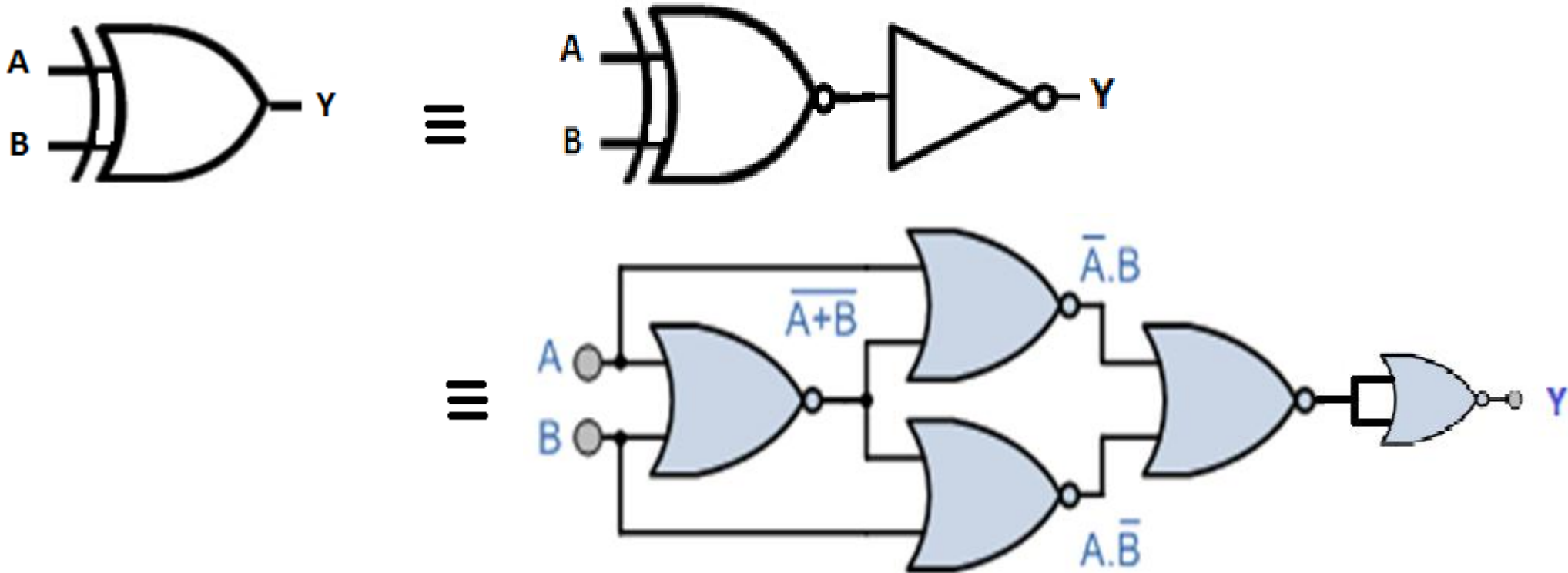
$$Y = (A'.B + A.B')' = (A'.B)' . (A.B')' = (A + B') . (A' + B)$$

$$Y = A.B + A'.B'$$

$$Y = A \odot B$$

## Realization of Boolean expression using Universal Gates

### ❖ XOR Gate:



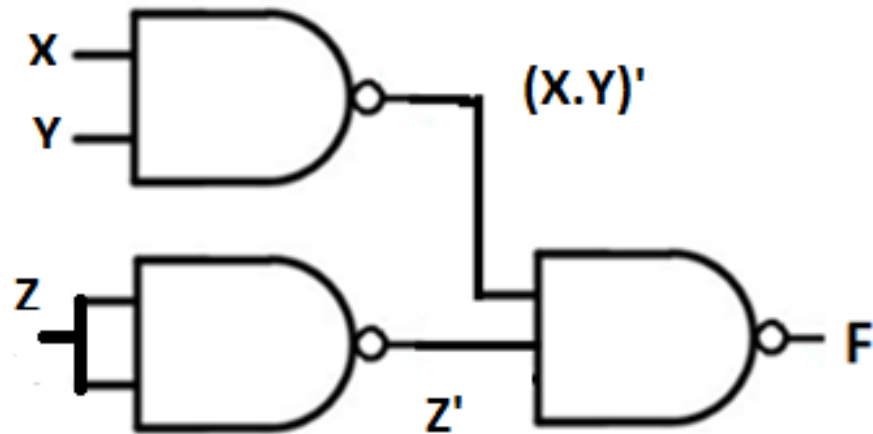
## Realization of Boolean expression using Universal Gates

❖ Implement the given function using NAND Gates only.

$$F = X.Y + Z$$

$$F = ((X.Y + Z)')'$$

$$F = ((X.Y)' . Z')'$$



## Problems on Boolean algebra

❖(1) Simplify the given Boolean expression and implement using basic gates

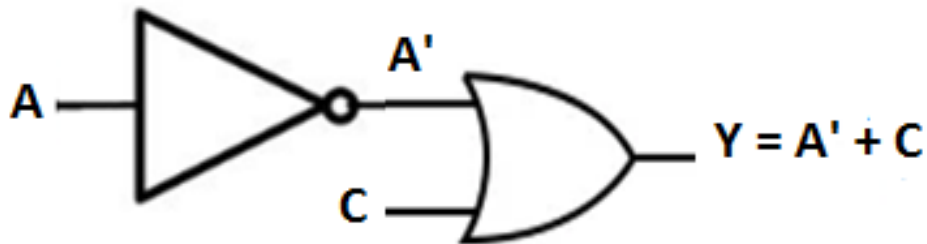
$$F = A.B.C + A' + A. B'.C$$

$$F = A.C (B + B') + A' \quad \text{where } B + B' = 1$$

$$F = A.C + A' \quad \text{Distributive Law}$$

$$F = (A' + A) . (A' + C) \quad A + B.C = (A+B) . (A+C)$$

$$F = A' + C$$



Logic Diagram

## Problems on Boolean algebra

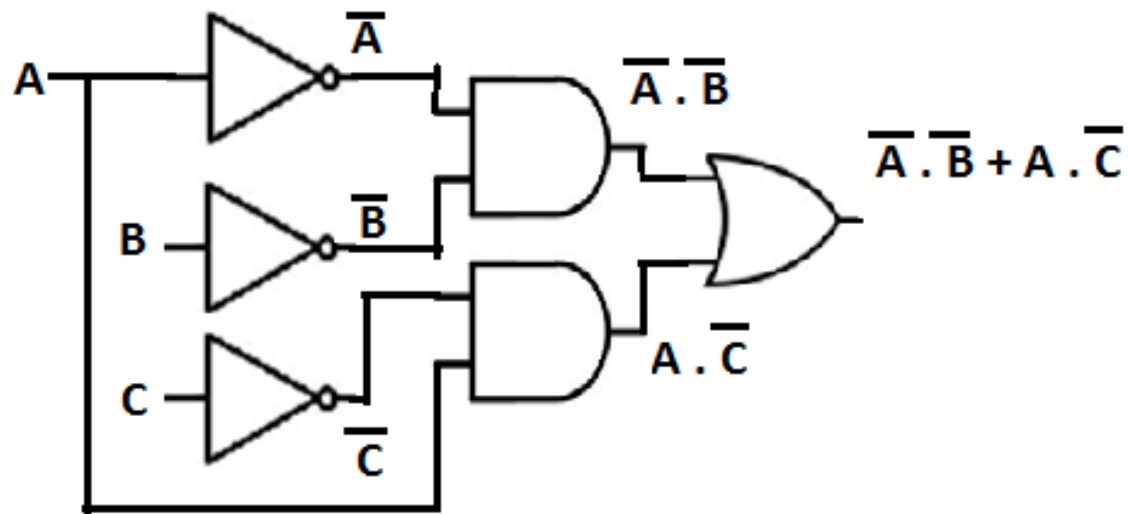
❖(2) Simplify and realize the given function using basic gates:

$$Y = A'B'C' + AB'C' + A'B' + AC'$$

$$Y = B'C' (A' + A) + A'B' + AC'$$

$$Y = B'C' \cdot (1) + A'B' + AC'$$

By Consensus Theorem:  $Y = A'B' + AC'$        $AB + \bar{A}C + BC = AB + \bar{A}C$



❖ (3) Simplify and Implement the given function using NAND Gates only.

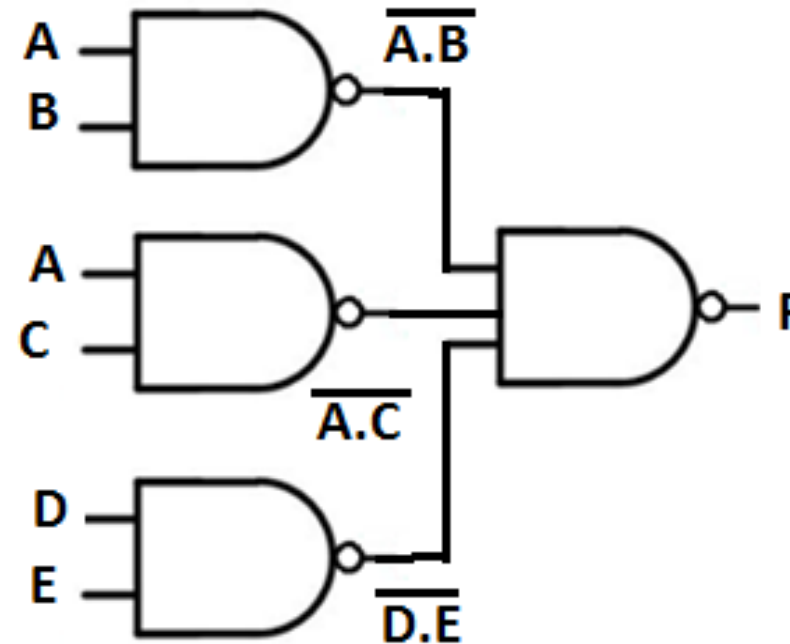
$$F = A(B + C) + DE$$

❖ SOP Form:  $F = AB + AC + DE$

$$F = \overline{\overline{AB + AC + DE}}$$

$$F = \overline{(\overline{A.B}) . (\overline{A.C}) . (\overline{D.E})}$$

DeMorgan's Law



Logic Diagram

## Problems on Boolean algebra

---

❖ (4) Simplify the given Boolean Expression:

$$Y = A + A'.B + A'.B'.C + A'.B'.C'.D + A'.B'.C'.D'.E$$

$$Y = A + A' (B + B'.C + B'.C'.D + B'.C'.D'.E)$$

Absorption Law:  $A + A'.B = A + B$

$$Y = A + B + B'.C + B'.C'.D + B'.C'.D'.E$$

$$Y = A + B + B' (C + C'.D + C'.D'.E)$$

$$Y = A + B + C + C'.D + C'.D'.E$$

$$Y = A + B + C + C' (D + D'.E)$$

$$Y = A + B + C + D + D'.E$$

$$\mathbf{Y = A + B + C + D + E}$$

## Problems on Boolean algebra

---

❖ (5) Evaluate the Boolean function  $Y = C + C'B + BA'$

when  $A = 1$ ,  $B = 0$  and  $C = 1$

$$Y = 1 + 1'.0 + 0.1' = 1 + 0 + 0$$

$$Y = 1$$

## Problems on Boolean algebra

---

- ❖ (6) Simplify the given Boolean expression and implement using Basic and NAND gates

$$F = AB + A(B+C) + B(B+C)$$

$$F = AB + AB + AC + BB + BC \quad \text{Distributive Law}$$

$$F = AB + AC + B + BC$$

$$F = AB + AC + B(1 + C)$$

$$F = AB + AC + B$$

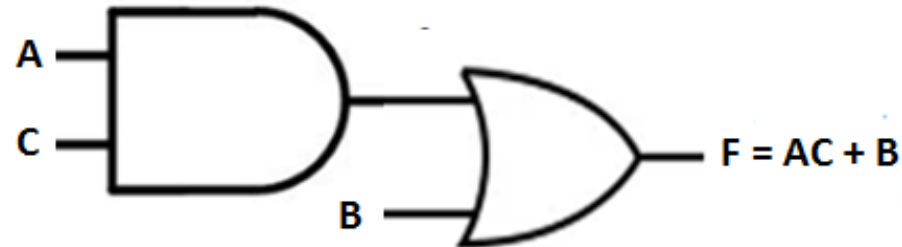
$$F = AC + B(A + 1)$$

$$F = AC + B$$

## Problems on Boolean algebra

❖  $F = A.C + B$

❖ Using Basic Gates:

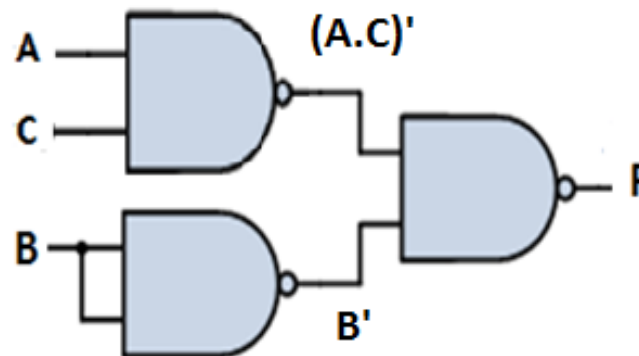


❖ Using NAND Gates:

$$F = AC + B$$

$$F = ((AC + B)')'$$

$$F = ((AC)' \cdot B')'$$



## Problems on Boolean algebra

---

- ❖ (7) Simplify and realize the given function using: (i) Basic Gates:  
(ii) NAND Gates

$$Y = (A'.B + A' + A.B)'$$

$$Y = (A'.B)' \cdot (A')' \cdot (A.B)'$$

De Morgan's Theorem

$$Y = ((A')' + B') \cdot (A) \cdot (A' + B')$$

$$Y = (A + B') \cdot ((A.A') + A.B')$$

$$Y = (A + B') \cdot (A.B')$$

$$Y = A.A.B' + A.B'.B'$$

$$Y = A.B' + A.B'$$

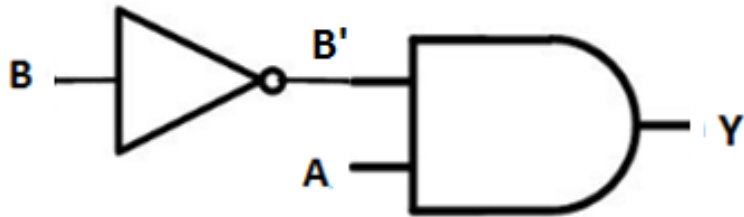
$$Y = A.B'$$

## Problems on Boolean algebra

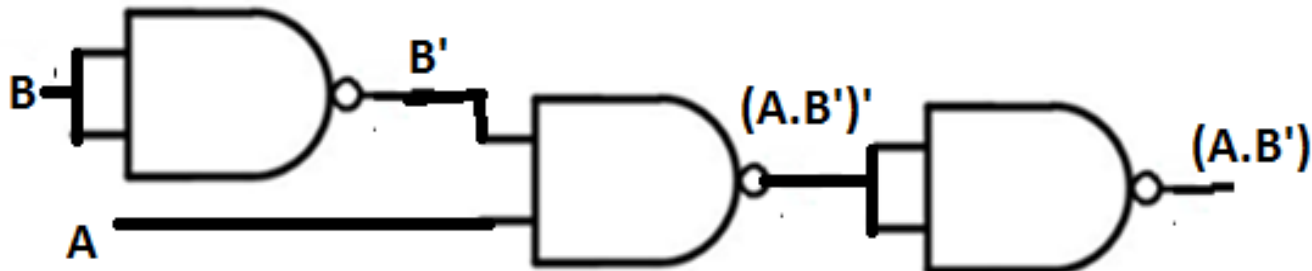
---

❖  $Y = A.B'$

❖ Using Basic Gates:



❖ Using NAND Gates:



## Problems on Boolean algebra

---

❖(8) Simplify the given expression

$$Y = AB' + (A' + B' + C.C')'$$

$$Y = AB' + (A' + B')'$$

$$Y = AB' + (A'' B'')$$

$$Y = AB' + (AB)$$

$$Y = A(B' + B)$$

$$Y = A$$

De Morgan's Theorem

## Problems on Boolean algebra

---

❖(9) Simplify the given expression

$$F = X'Y'Z + XYZ + X'YZ + XY'Z$$

$$F = Y'Z(X + X') + YZ(X' + X)$$

$$F = Y'Z + YZ$$

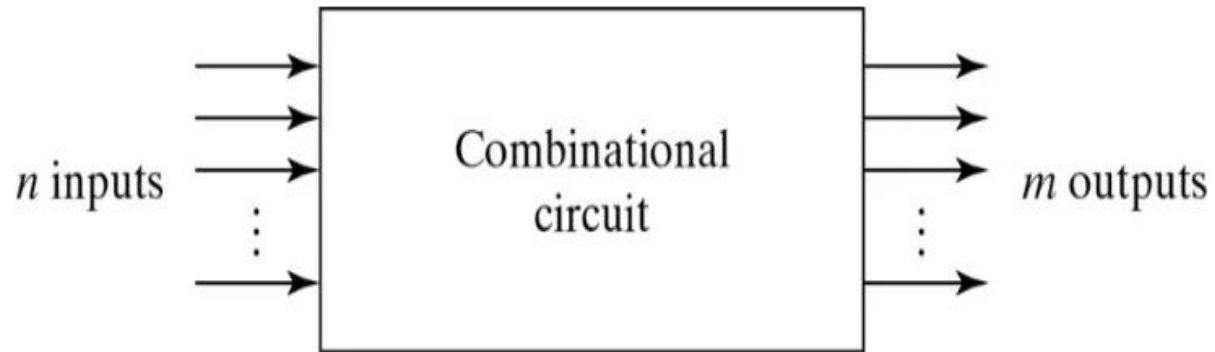
$$F = Z(Y' + Y)$$

$$Y = Z$$

## Combinational Logic Circuits: Half Adder and Full adder

---

- ❖ **Combinational circuits** are constructed by interconnection of logic gates. whose outputs at any time are determined from only the present combination of inputs



- ❖ A combinational circuit performs an operation that can be specified logically by a set of **Boolean functions**
- ❖ Examples: Binary Adders, Multiplexers, etc.

## Combinational Logic Circuits: Half Adder and Full adder

### ❖ Half Adder

- x and y are the two binary inputs
- Sum (s) and Carry (c) are the two binary outputs

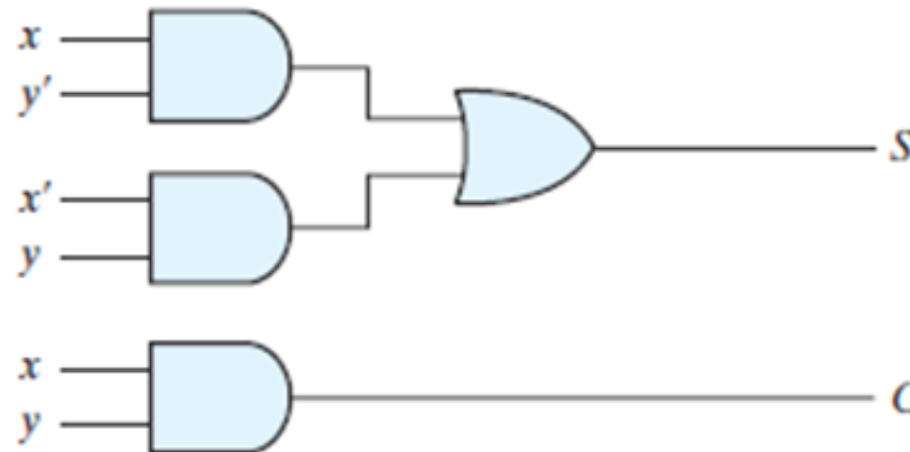
Boolean Expression:

$$s = f(x, y) = \sum(1, 2) = x'.y + x.y'$$

$$c = f(x, y) = \sum(3) = x.y$$

Truth Table

| <i>x</i> | <i>y</i> | <i>C</i> | <i>S</i> |
|----------|----------|----------|----------|
| 0        | 0        | 0        | 0        |
| 0        | 1        | 0        | 1        |
| 1        | 0        | 0        | 1        |
| 1        | 1        | 1        | 0        |



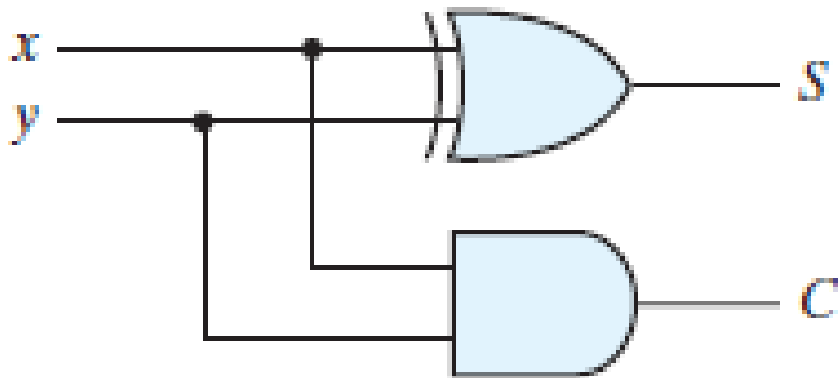
Logic Diagram

## Combinational Logic Circuits: Half Adder and Full adder

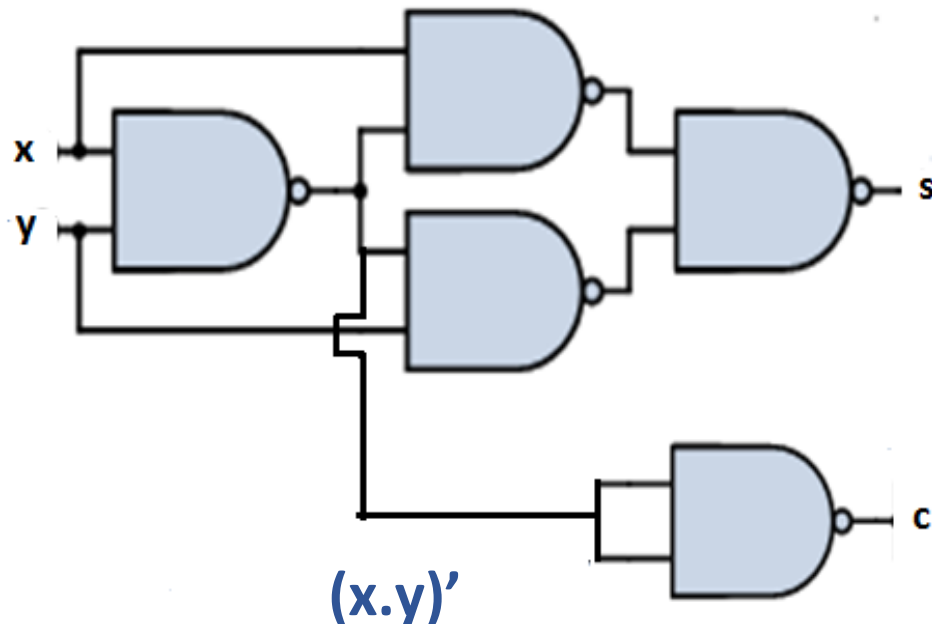
❖ **Half Adder:**  $s = x \oplus y$   
 $c = x.y$

Sum expression:  $s = x'.y + x.y'$

❖ **Half Adder using Logic Gates**



❖ **Half Adder using NAND Gates only**



## Combinational Logic Circuits: Half Adder and Full adder

### ❖ Full Adder:

- x, y and z are three binary inputs.
- S is sum and C is carry outputs

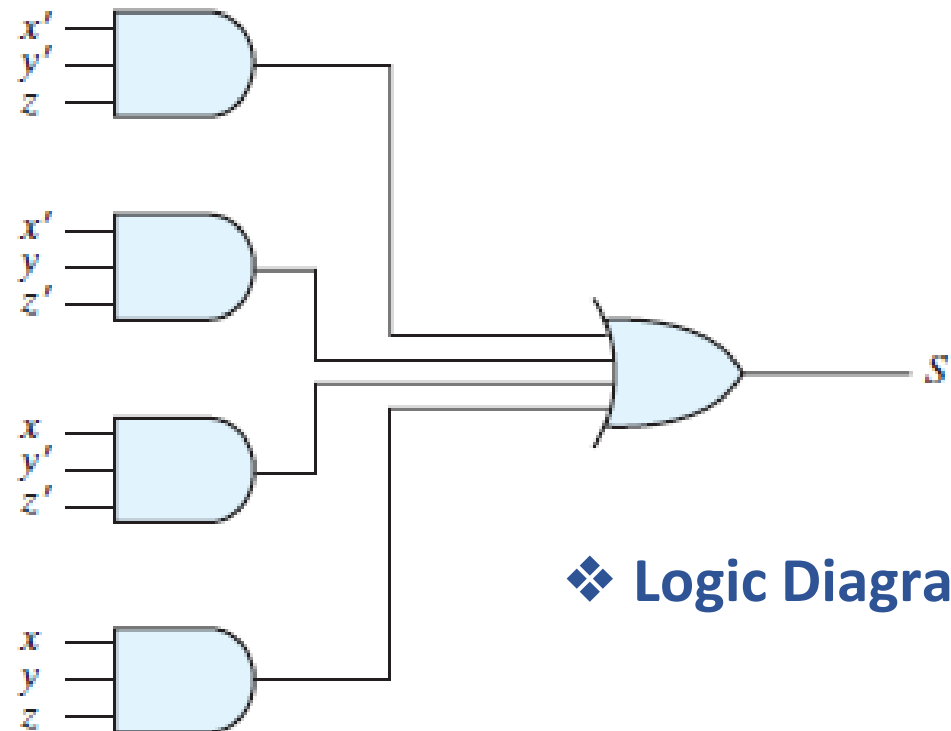
### ❖ Truth Table:

| <i>x</i> | <i>y</i> | <i>z</i> | <i>C</i> | <i>S</i> |
|----------|----------|----------|----------|----------|
| 0        | 0        | 0        | 0        | 0        |
| 0        | 0        | 1        | 0        | 1        |
| 0        | 1        | 0        | 0        | 1        |
| 0        | 1        | 1        | 1        | 0        |
| 1        | 0        | 0        | 0        | 1        |
| 1        | 0        | 1        | 1        | 0        |
| 1        | 1        | 0        | 1        | 0        |
| 1        | 1        | 1        | 1        | 1        |

### ❖ Boolean Expression:

$$S = f(x, y, z) = \sum(1, 2, 4, 7) = x'y'z + x'yz' + xy'z' + xyz$$

$$C = f(x, y, z) = \sum(3, 5, 6, 7) = x'yz + xy'z + xyz' + xyz$$



### ❖ Logic Diagram

## Combinational Logic Circuits: Half Adder and Full adder

### ❖ Full Adder:

#### ➤ Carry Expression:

$$C = x'yz + xy'z + xyz' + xyz$$

$$C = x'yz + xy'z + xy(z' + z)$$

$$C = x'yz + xy'z + xy$$

$$C = x'yz + x(y'z + y) \quad \text{Absorption Law}$$

$$C = x'yz + x(z + y)$$

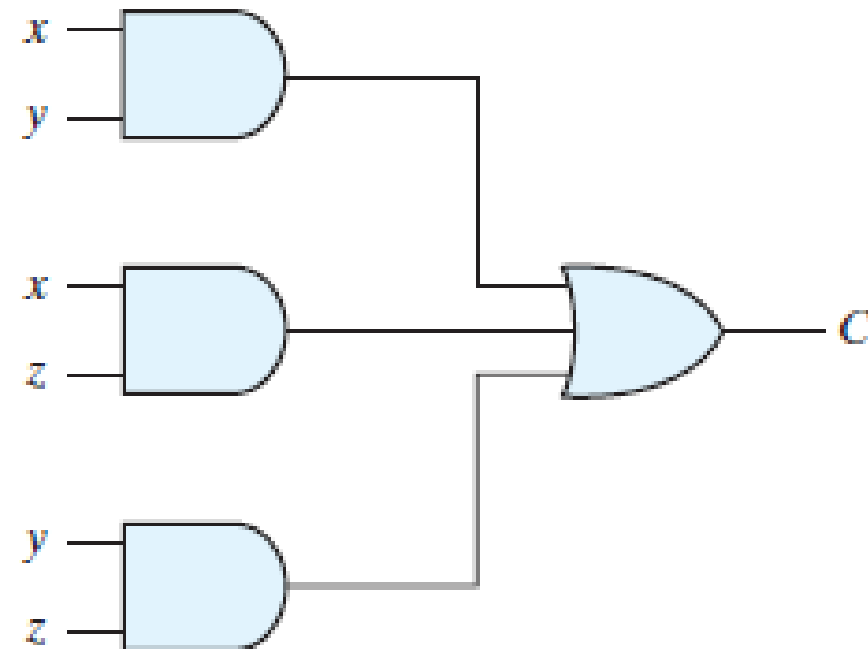
$$C = x'yz + xz + xy$$

$$C = z(x'y + x) + xy$$

$$C = z(y + x) + xy$$

$$\mathbf{C = yz + xz + xy}$$

### ❖ Logic Diagram



## Implement Full adder using two Half Adder

---

### ❖ Boolean Expression for Sum:

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$S = x' (y'z + yz') + x (y'z' + yz)$$

$$S = x' (y \oplus z) + x ((y \oplus z)')$$

$$S = x \oplus (y \oplus z)$$

### ❖ Boolean Expression for Carry:

$$C = x'yz + xy'z + xyz' + xyz$$

$$C = z. (x'y + xy') + xy (z' + z)$$

$$C = z. (x \oplus y) + xy .(1)$$

$$C = (x \oplus y).z + xy$$

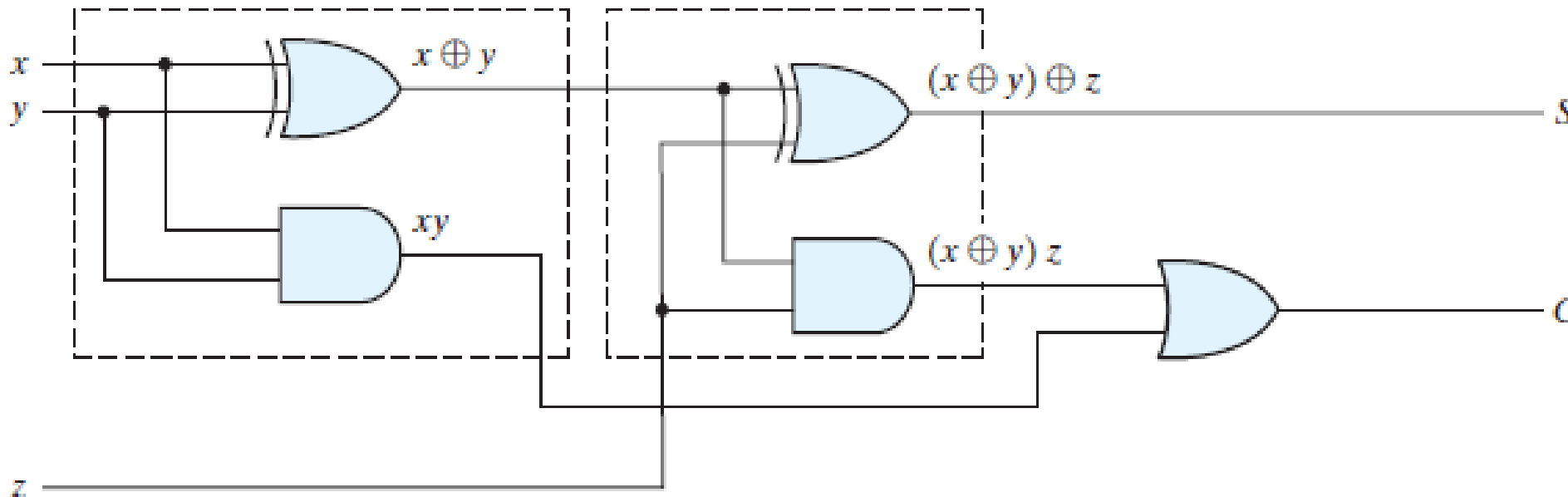
## Implement Full adder using two Half Adder

❖ Full Adder Boolean Expression:

$$S = (x \oplus y) \oplus z$$

$$C = (x \oplus y).z + x.y$$

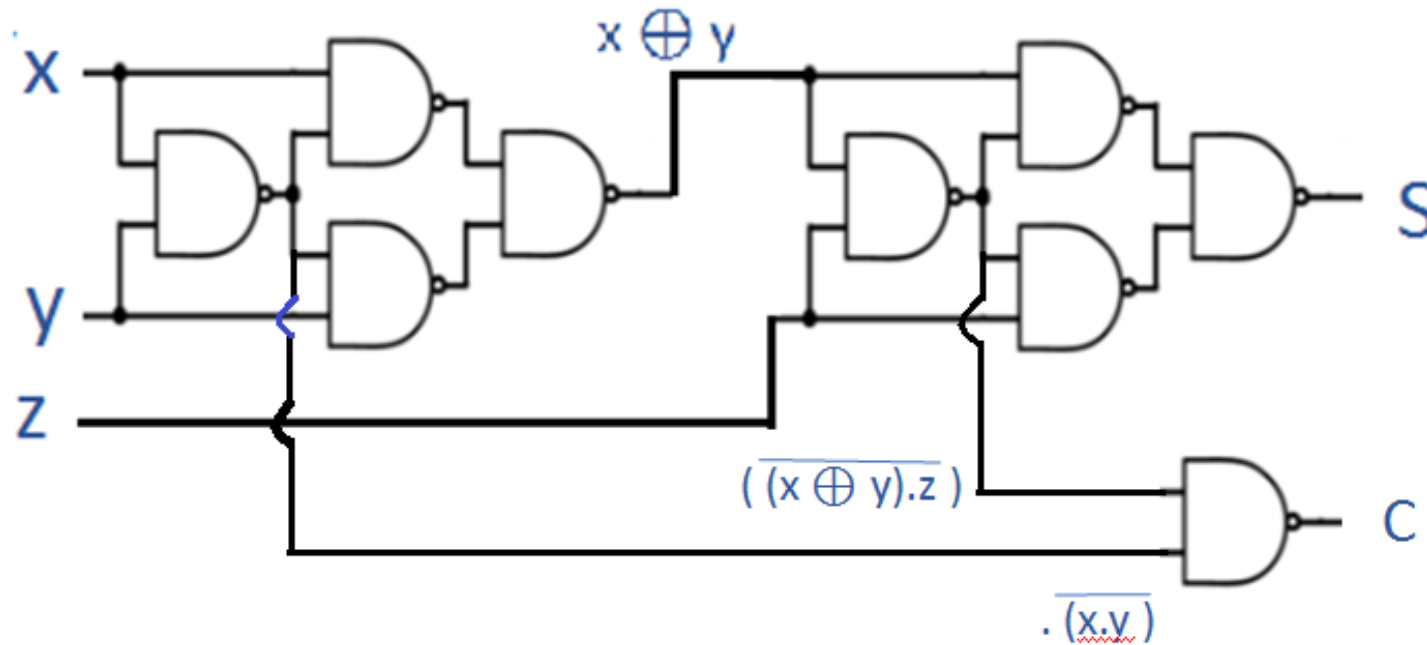
❖ Implementation of full adder with two half adders and an OR gate



## Implement Full adder using NAND only

- ❖ Boolean Expression:  $S = (x \oplus y) \oplus z$   
 $C = (x \oplus y).z + x.y$

- ❖ Full adder circuit using NAND Gates



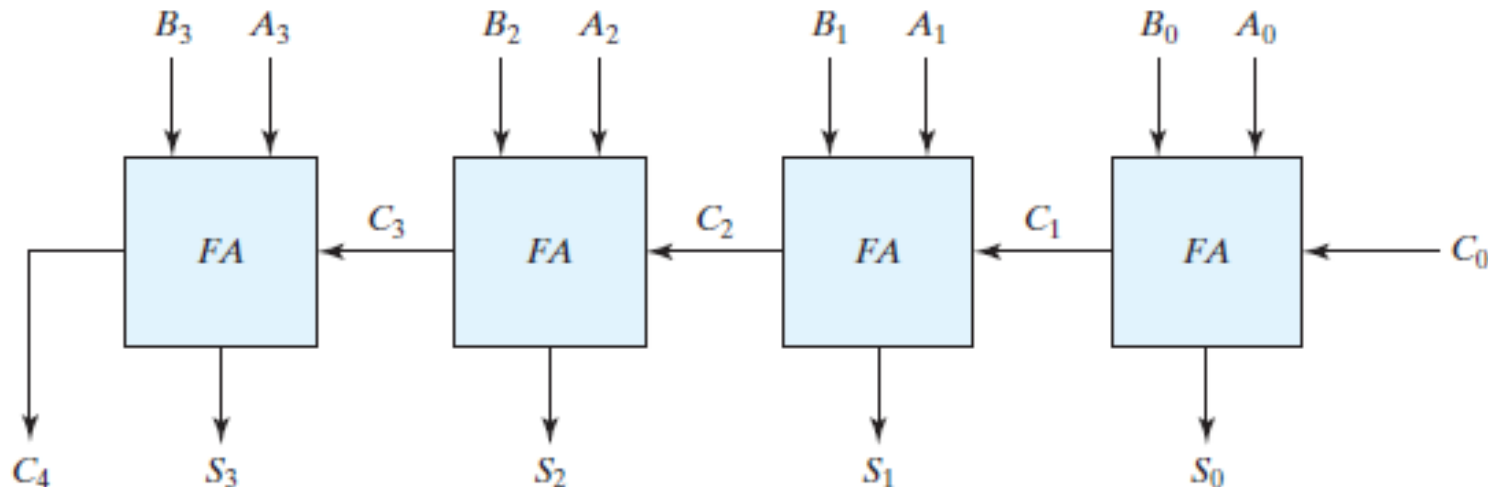
$$C = \overline{\overline{((x \oplus y).z + x.y)}}$$

$$C = \overline{\overline{((x \oplus y).z)} \cdot \overline{(x.y)}}$$

## Combinational Logic Circuits: Half Adder and Full adder

### ❖ Four-bit adder

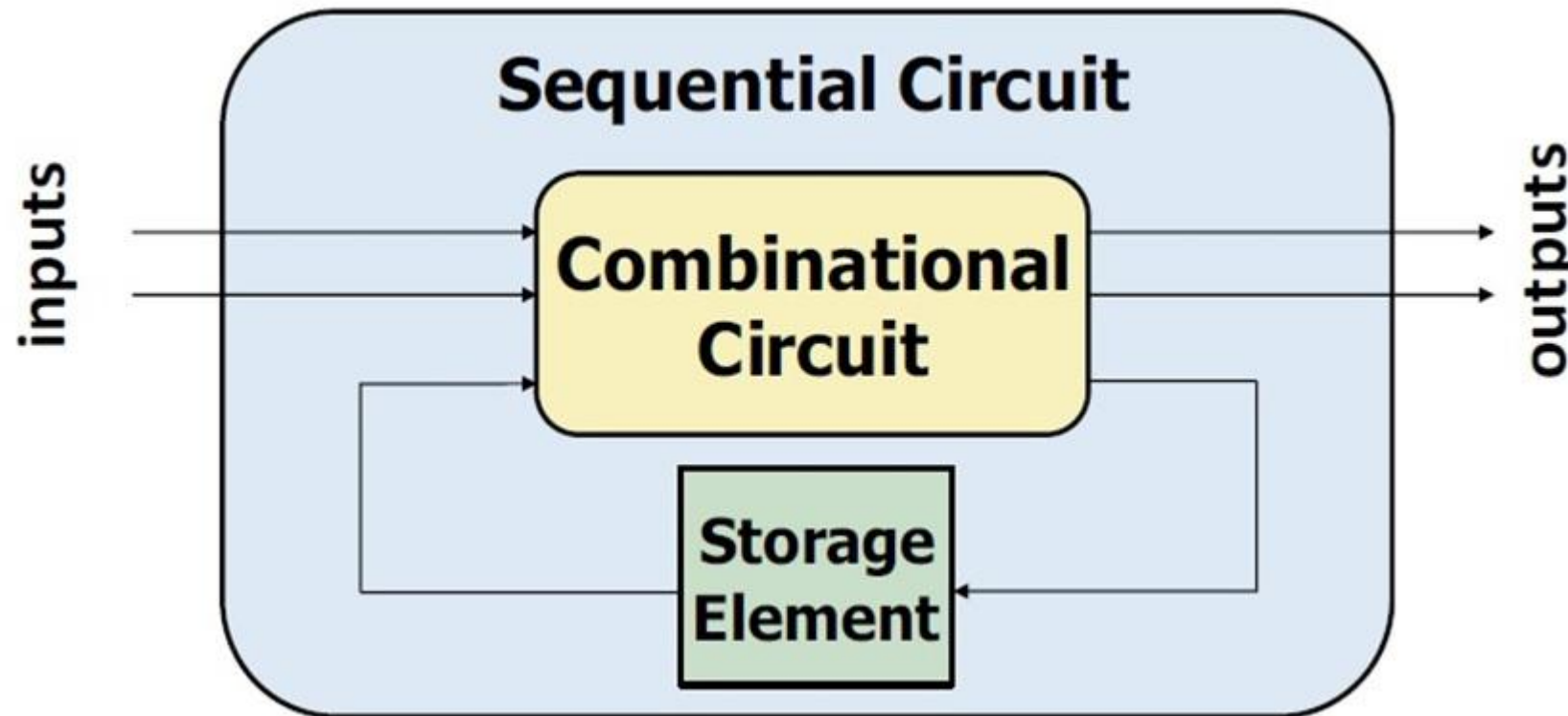
Using four Full adder Ripple adder circuit is constructed.



### ❖ Example

$$\begin{array}{r} 1110 \\ A = 1101 \\ + B = 0111 \\ \hline 10100 \end{array}$$

- We want circuits that produce the output depending on the **current** and **past** input values - Circuits with **memory**.
- How do we design such a circuit that **stores information**?



- Sequential Circuits are of **two** types

### 1. Synchronous Circuits:

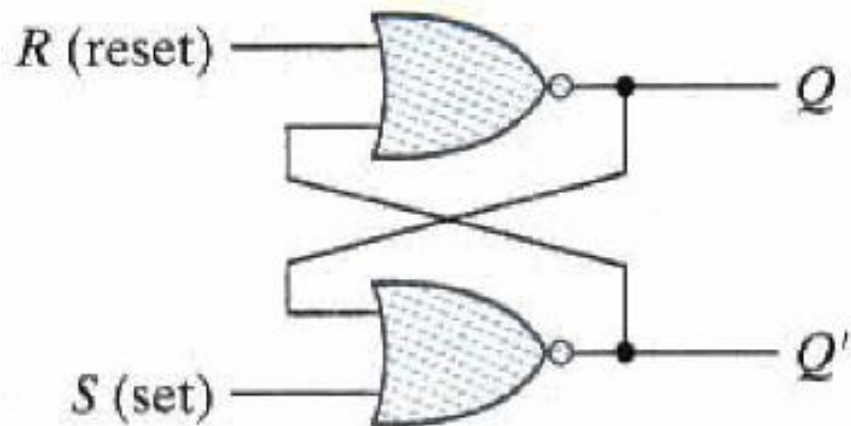
- ☐ In synchronous sequential circuits, the state of the device changes only at discrete times in response to a **clock pulse**.

### 2. Asynchronous Circuits:

- ☐ Asynchronous circuit is **not synchronized** by a clock signal; the outputs of the circuit change directly in response to the change in the inputs.

- *Storage elements that operate with signal levels (rather than signal transitions) are referred to as **latches** ; those controlled by a clock transition are **flip-flops** .*
- Latches are said to be **level sensitive** devices; flip-flops are **edge-sensitive** devices.
- The two types of storage elements are related because **latches are the basic circuits** from which all flip-flops are constructed.
- Although latches are useful for storing binary information and for the design of asynchronous sequential circuits, **they are not practical for use** as storage elements in synchronous sequential circuits.

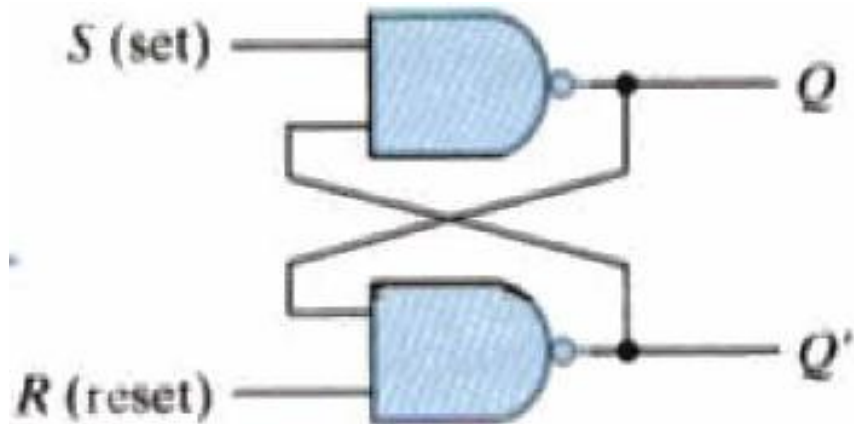
### Circuit Diagram



### Function Table

| $S$ | $R$ | $Q$ | $Q'$ |                         |
|-----|-----|-----|------|-------------------------|
| 1   | 0   | 1   | 0    |                         |
| 0   | 0   | 1   | 0    | (after $S = 1, R = 0$ ) |
| 0   | 1   | 0   | 1    |                         |
| 0   | 0   | 0   | 1    | (after $S = 0, R = 1$ ) |
| 1   | 1   | 0   | 0    | (forbidden)             |

### Circuit Diagram



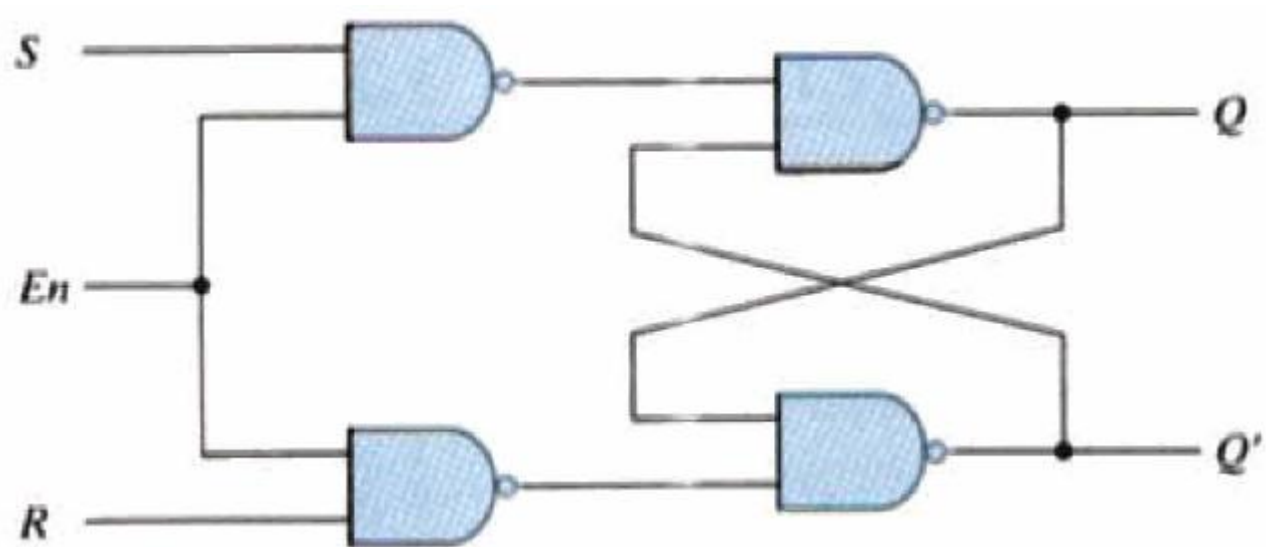
### Function Table

| $S$ | $R$ | $Q$ | $Q'$                      |
|-----|-----|-----|---------------------------|
| 1   | 0   | 0   | 1                         |
| 1   | 1   | 0   | 1 (after $S = 1, R = 0$ ) |
| 0   | 1   | 1   | 0                         |
| 1   | 1   | 1   | 0 (after $S = 0, R = 1$ ) |
| 0   | 0   | 1   | 1 (forbidden)             |

# Electronic Principles and Devices

## The S-R (Set - Reset) latch with control input

### Logic Diagram



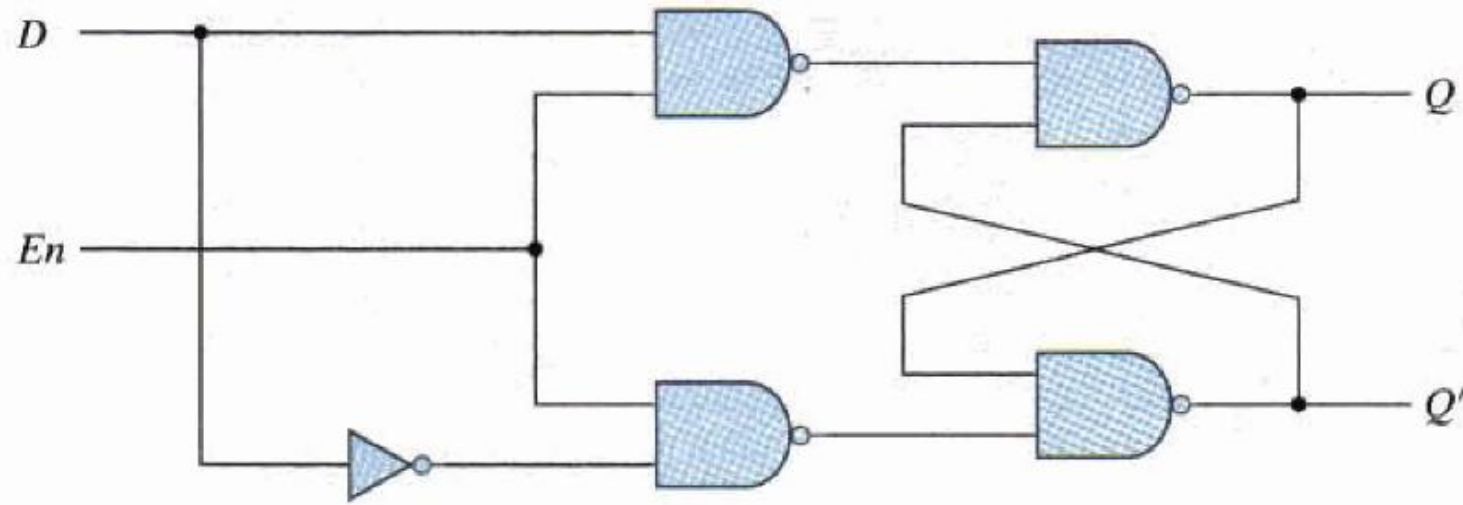
### Function Table

| $En$ | $S$ | $R$ | Next state of $Q$     |
|------|-----|-----|-----------------------|
| 0    | X   | X   | No change             |
| 1    | 0   | 0   | No change             |
| 1    | 0   | 1   | $Q = 0$ ; reset state |
| 1    | 1   | 0   | $Q = 1$ ; set state   |
| 1    | 1   | 1   | Indeterminate         |

# Electronic Principles and Devices

## The D latch (Transparent latch)

### Logic Diagram



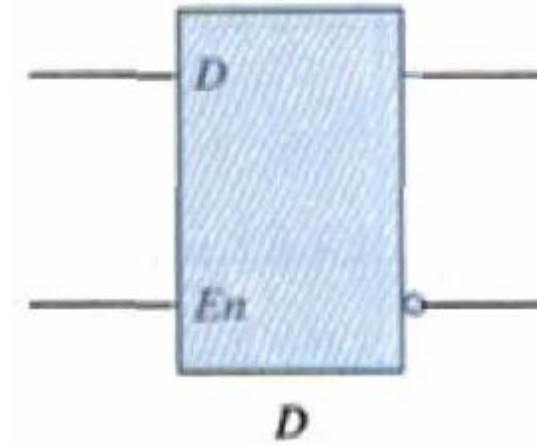
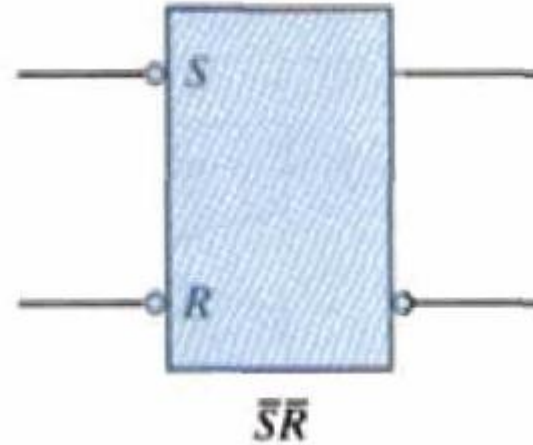
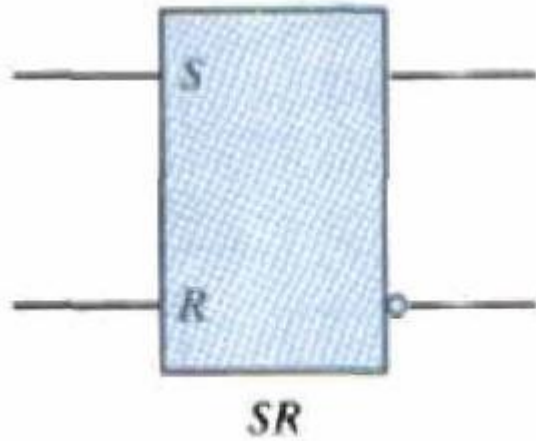
### Function Table

| $En$ | $D$ | Next state of $Q$     |
|------|-----|-----------------------|
| 0    | X   | No change             |
| 1    | 0   | $Q = 0$ ; reset state |
| 1    | 1   | $Q = 1$ ; set state   |

# Electronic Principles and Devices

## Graphic symbols for latches

---



- The state of a latch or flip-flop is switched by a change in the control input.
- This momentary change is called a *trigger*, and the transition it causes is said to trigger the flip-flop.
- The *D* latch with pulses in its control input is essentially a flip-flop that is triggered every time the pulse goes to the logic-1 level.
- As long as the pulse input remains at this level, any changes in the data input will change the output and the state of the latch.

- Flip-flop circuits are constructed in such a way as to make them operate properly when they are part of a sequential circuit that employs a common clock.
- The problem with the latch is that it responds to a change in the *level* of a clock pulse.
- As shown in Fig. (a), a positive level response in the enable input allows changes in the output when the  $D$  input changes while the clock pulse stays at logic 1.
- The key to the proper operation of a flip-flop is to trigger it only during a signal *transition* .



(a) Response to positive level



(b) Positive-edge response



(c) Negative-edge response

**Latch will change the Output when the clock remains as logic 1 as shown in figure (a)**

**Flip-Flops will change the Output when the clock changes from logic 0 to Logic 1 as shown in figure (b)**

- Construction of an edge-triggered  $D$  flip-flop uses three  $SR$  latches as shown in Fig.
- Two latches respond to the external  $D$  (data) and  $Clk$  (clock) inputs. The third latch provides the outputs for the flip-flop.
- The  $S$  and  $R$  inputs of the output latch are maintained at the logic-1 level when  $Clk = 0$ .
- This causes the output to remain in its present state. Input  $D$  may be equal to 0 or 1.
- If  $D = 0$  when  $Clk$  becomes 1,  $R$  changes to 0. This causes the flip-flop to go to the reset state, making  $Q = 0$ . If there is a change in the  $D$  input while  $Clk = 1$ , terminal  $R$  remains at 0 because  $Q$  is 0.
- Thus, the flip-flop is locked out and is unresponsive to further changes in the input.

- Similarly, if  $D = 1$  when  $Clk$  goes from 0 to 1,  $S$  changes to 0.
- This causes the circuit to go to the set state, making  $Q = 1$ .
- Any change in  $D$  while  $Clk = 1$  does not affect the output.

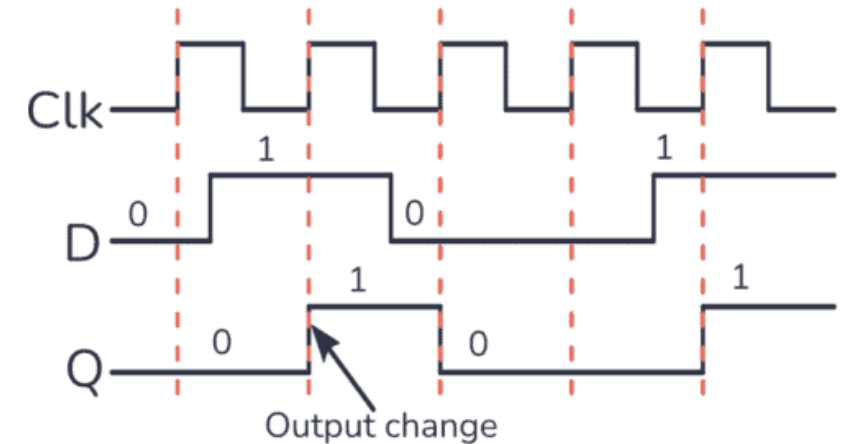
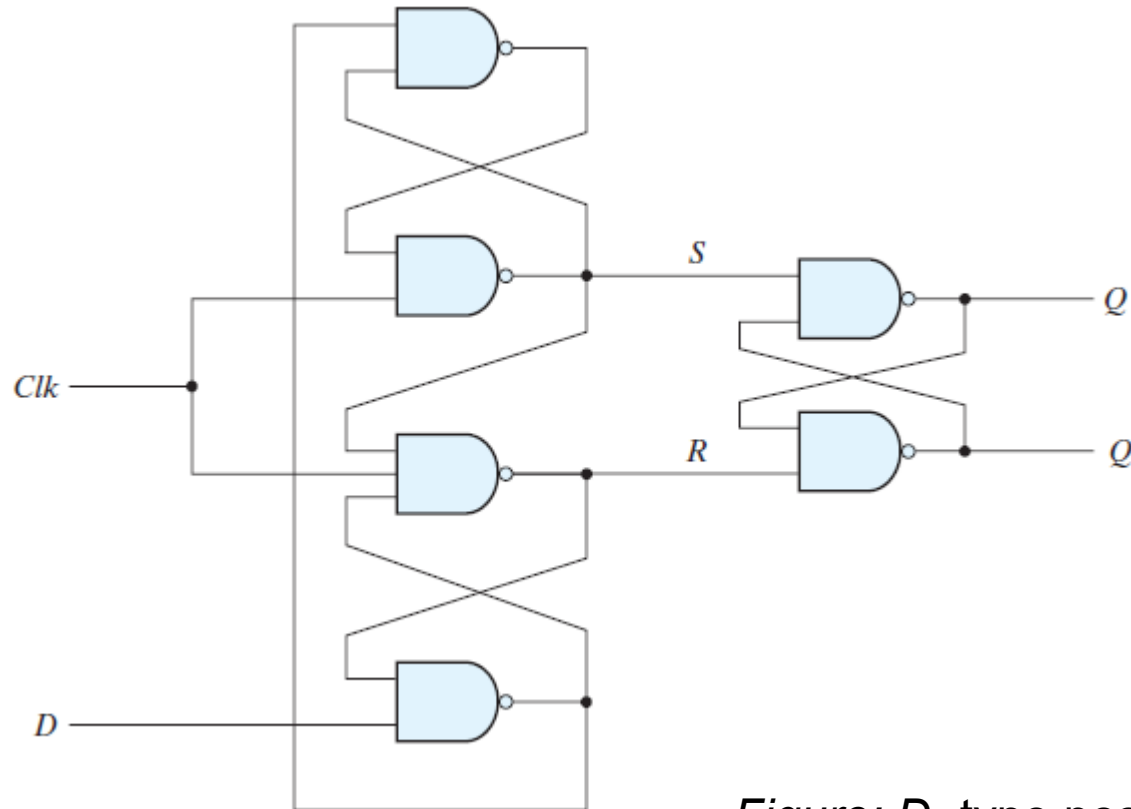
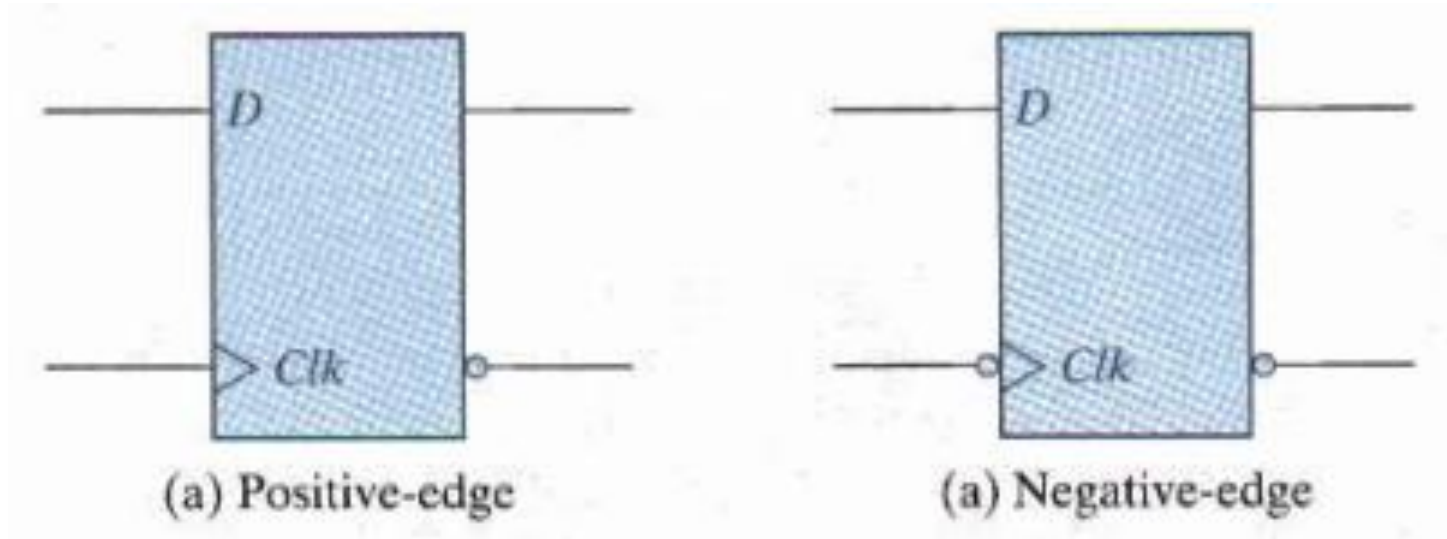


Figure: D -type positive-edge-triggered flip-flop

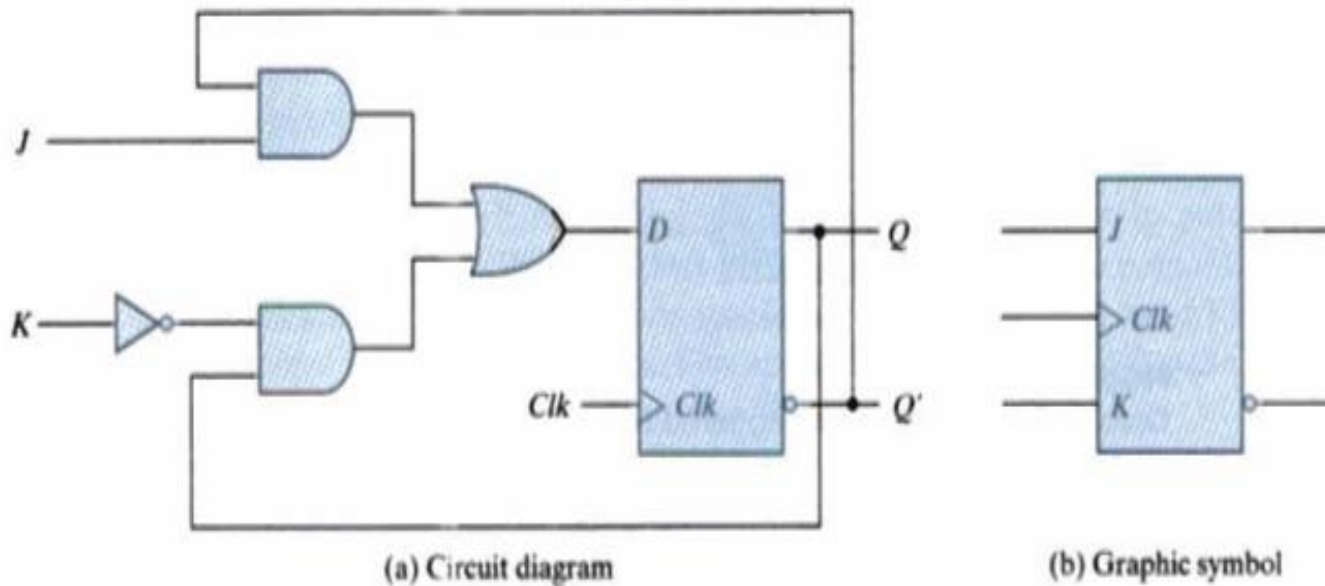


In D Flip-Flops, **Absence** of the **bubble** in front of the **Clk** indicates **Positive** edge triggered as shown in figure (a)

In D Flip-Flops, **Presence** of the **bubble** in front of the **Clk** indicates **Negative** edge triggered as shown in figure (b)

# Electronic Principles and Devices

## JK Flip – Flop using D flip flop



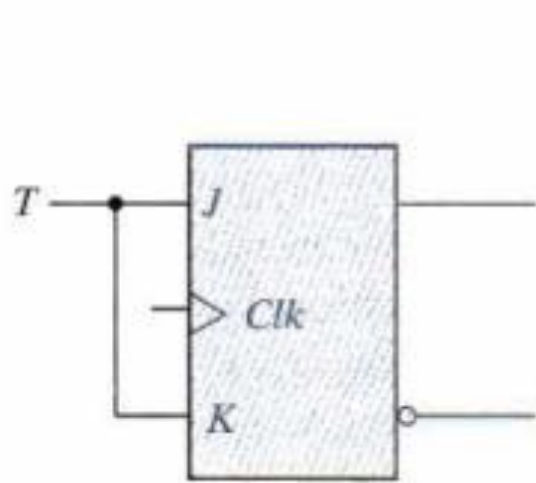
*Flip-Flop Characteristic Tables*

**JK Flip-Flop**

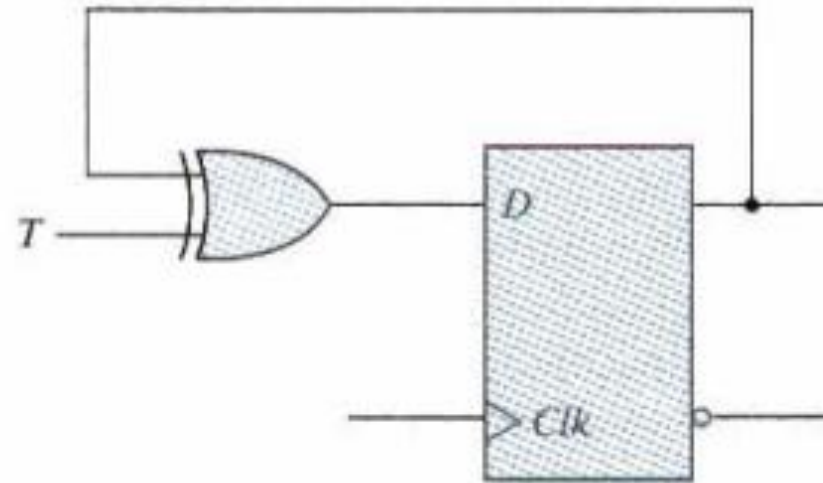
| <i><b>J</b></i> | <i><b>K</b></i> | <i><b>Q(t + 1)</b></i> |            |
|-----------------|-----------------|------------------------|------------|
| 0               | 0               | $Q(t)$                 | No change  |
| 0               | 1               | 0                      | Reset      |
| 1               | 0               | 1                      | Set        |
| 1               | 1               | $Q'(t)$                | Complement |

- The Inputs are **J-K** along with the Clk
  - J-K inputs comes after its inventors **Jack Kilby**.

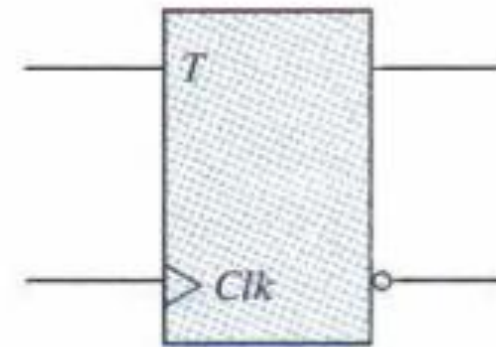
- **JK flip-flop** is a sequential circuit, whose state transitions are **synchronized** with the **clock pulse** (+ve / -ve ) edge triggered.
- In **JK flip-flop** all the rows are a valid state in the characteristic table, also when  $J=K=1$ , the output toggles, **Toggle state**.
- **D flip-flop** is a sequential circuit, whose output is same as the input and hence also called as **transparent flip-flop**.



(a) From JK flip-flop



(b) From D flip-flop



(c) Graphic symbol

## T Flip-Flop using JK Flip-Flop & D Flip-Flop

- T Flip-Flop is a **synchronous** device, where high to low or low to high transitions is passed through **clock signal** which **changes** the output state of Flip-Flop.

| <b>T Flip-Flop</b> |                 |            |
|--------------------|-----------------|------------|
| <b>T</b>           | <b>Q(t + 1)</b> |            |
| 0                  | $Q(t)$          | No change  |
| 1                  | $Q'(t)$         | Complement |

### Advantages of T flip-flop:

- These Flip-Flops has a **toggle input** and a **clock**.
- They are used for designing the **counters**.

- A *register* is a group of flip-flops, each one of which shares a common clock and is capable of storing one bit of information.
- An  $n$ -bit register consists of a group of  $n$  flip-flops capable of storing  $n$  bits of binary information.
- In addition to the flip-flops, a register may have combinational gates that perform certain data-processing tasks.
- In its broadest definition, a register consists of a group of flip-flops together with gates that affect their operation.
- The flip-flops hold the binary information, and the gates determine how the information is transferred into the register.

# Electronic Principles and Devices

## 4-bit Register:

---

- Various types of registers are available commercially.
- The simplest register is one that consists of only flip-flops, without any gates.
- Following Figure (next slide) shows such a register constructed with four *D*-type flip-flops to form a four-bit data storage register.
- The common clock input triggers all flip-flops on the positive edge of each pulse, and the binary data available at the four inputs are transferred into the register.
- The four outputs can be sampled at any time to obtain the binary information stored in the register.
- The input Clear goes to the active-low R (reset) input of all four flip-flops. When this input goes to 0, all flip-flops are reset asynchronously.
- The R inputs must be maintained at logic 1 (i.e., de-asserted) during normal clocked operation.

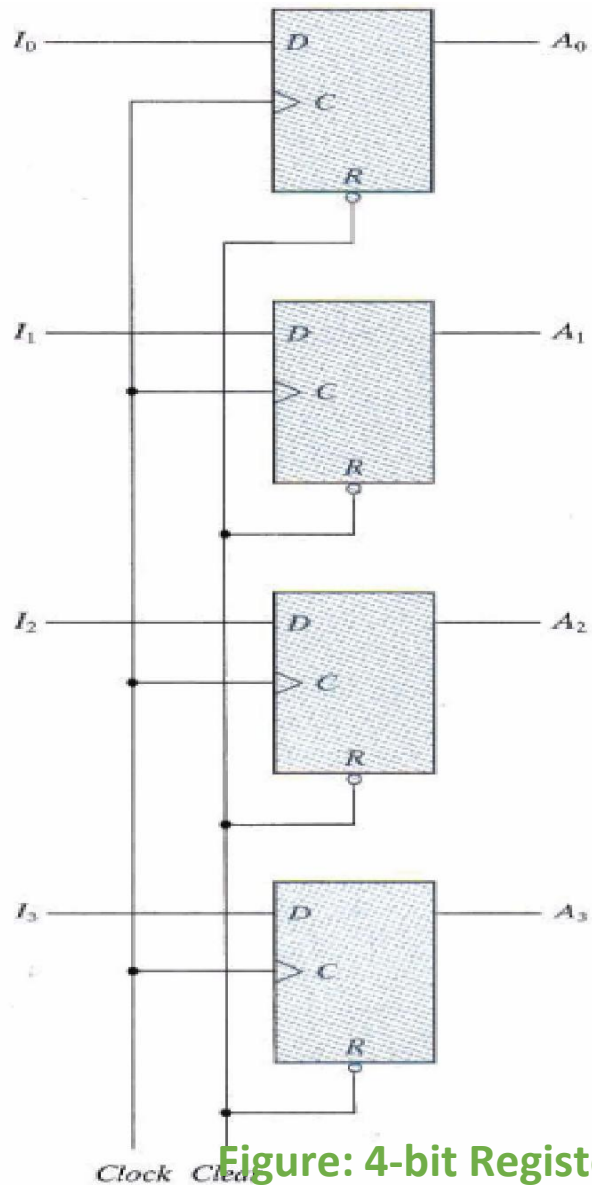
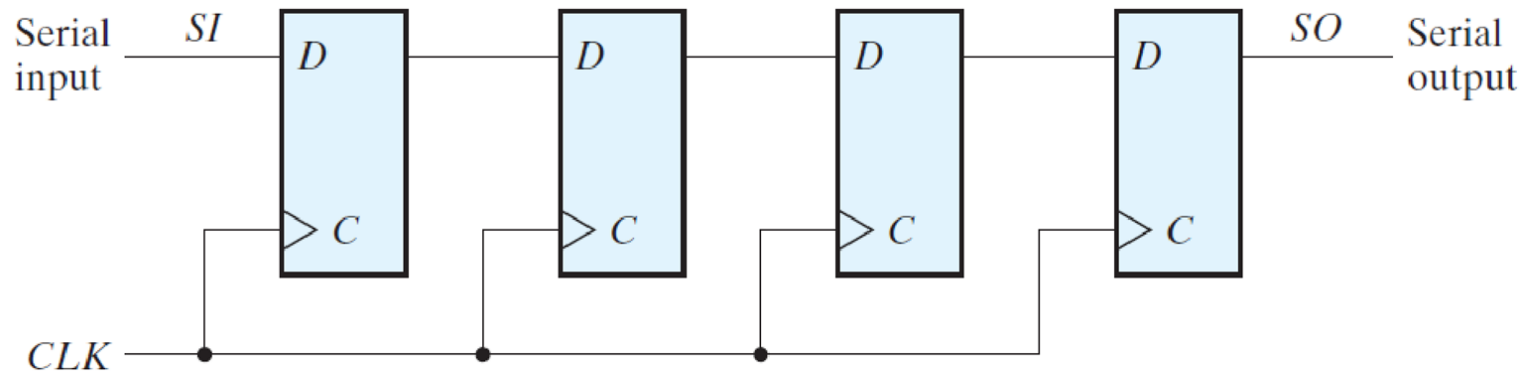


Figure: 4-bit Register

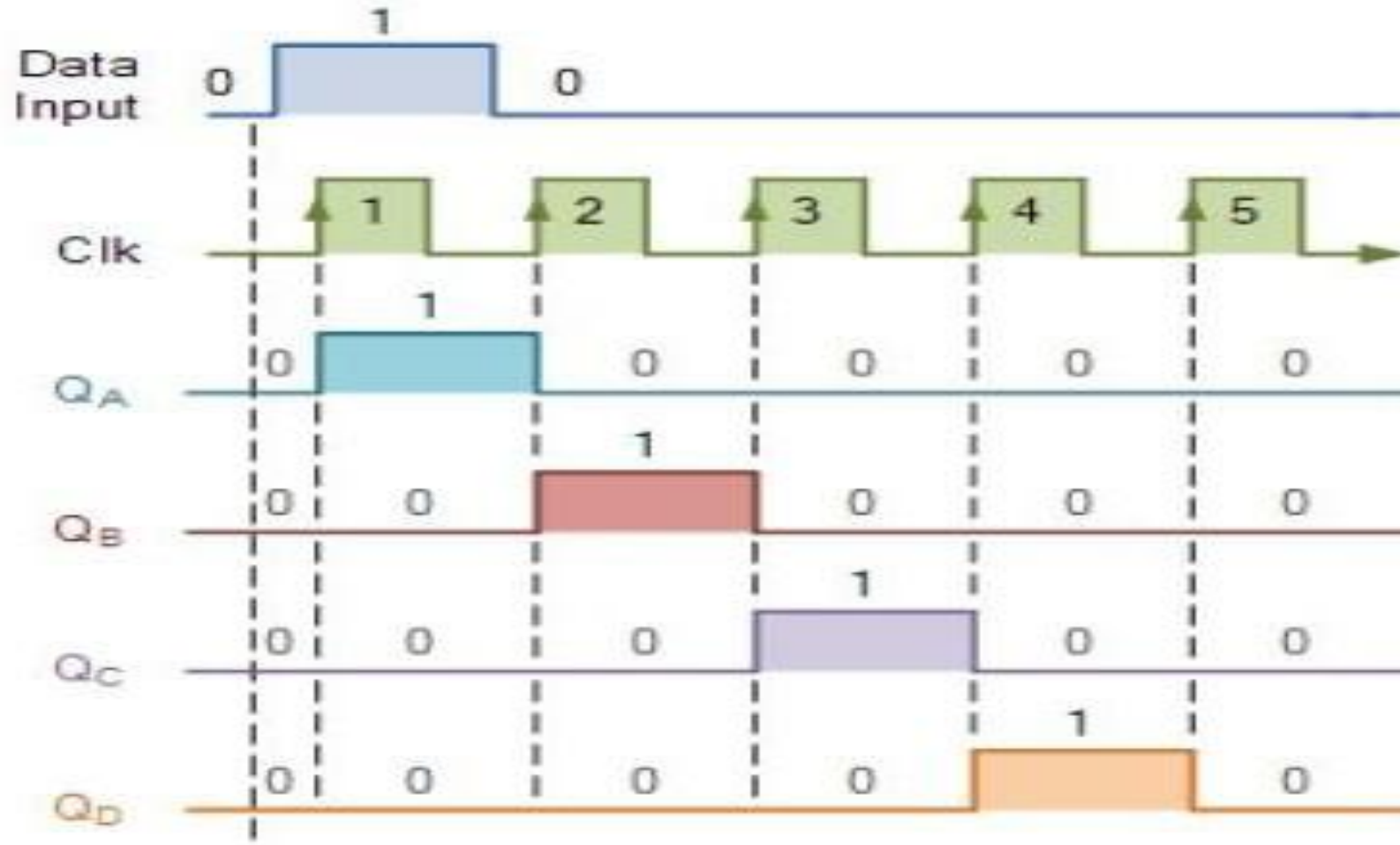
- The **transfer** of new information into a register is referred to as **loading or updating** the register.
- If **all the bits** of the register are loaded **simultaneously** with a common clock pulse, we say that the **loading is done in parallel**.
- A **clock edge applied to the C inputs** of the register of Figure, will load all four inputs in parallel.
- In this configuration, if the contents of the register must be left unchanged, the inputs must be held constant or the clock must be inhibited from the circuit.

- A **register capable of shifting** the binary information held in each cell to its neighboring cell, in a selected direction, is called a **shift register**.
- The **logical configuration** of a shift register consists of a chain of flip-flops in cascade, with the output of one flip-flop connected to the input of the next flip-flop.
- All **flip-flops receive common clock pulses**, which activate the shift of data from one stage to the next.
- There are **four different types** of shift registers:
  - Serial In Serial Out (SISO)
  - Serial In Parallel Out (SIPO)
  - Parallel In Serial Out (PISO)
  - Parallel In Parallel Out (PIPO)

- The **simplest possible shift register (SISO)** is one that uses only flip-flops, as shown in the following Figure .
- The output of a given flip-flop is connected to the *D* input of the flip-flop at its right.
- This shift register is **unidirectional** (left-to-right).
- Each clock pulse shifts the contents of the register one bit position to the right.



**FIGURE:** Four-bit shift register



Timing Diagram for SISO register

# Electronic Principles and Devices

## 4-bit Serial Input Serial Output (SISO) register:

| Clock Pulse No | QA | QB | QC | QD |
|----------------|----|----|----|----|
| 0              | 0  | 0  | 0  | 0  |
| 1              | 1  | 0  | 0  | 0  |
| 2              | 0  | 1  | 0  | 0  |
| 3              | 0  | 0  | 1  | 0  |
| 4              | 0  | 0  | 0  | 1  |
| 5              | 0  | 0  | 0  | 0  |

**Data movement through a shift register**

# Electronic Principles and Devices

## 4-bit Serial Input Serial Output (SISO) register:

### Example:

The contents of a four-bit register is initially 0110. The register is shifted six times to the right with the serial input being 1011100. What is the content of the register after each shift?

### Answer:

0110; 0011; 0001; 1000; 1100; 1110; 0111; 1011

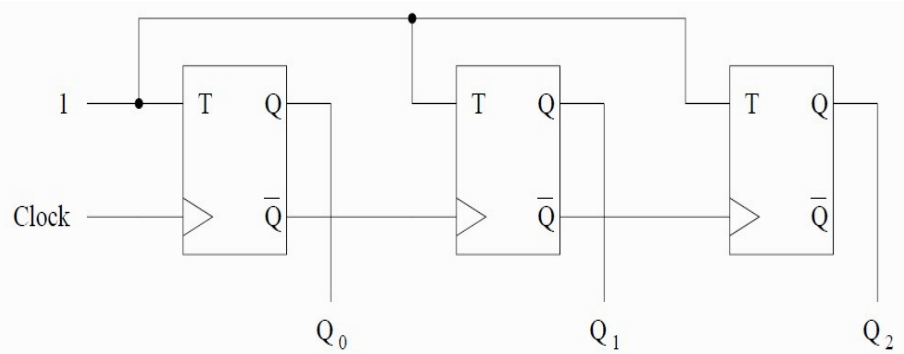
|                 | Data    | QA | QB | QC | QD |
|-----------------|---------|----|----|----|----|
|                 | initial | 0  | 1  | 1  | 0  |
| 1 <sup>st</sup> | 0 →     | 0  | 0  | 1  | 1  |
| 2 <sup>nd</sup> | 0 →     | 0  | 0  | 0  | 1  |
| 3 <sup>rd</sup> | 1 →     | 1  | 0  | 0  | 0  |
| 4 <sup>th</sup> | 1 →     | 1  | 1  | 0  | 0  |
| 5 <sup>th</sup> | 1 →     | 1  | 1  | 1  | 0  |
| 6 <sup>th</sup> | 0 →     | 0  | 1  | 1  | 1  |
| 7 <sup>th</sup> | 1 →     | 1  | 0  | 1  | 1  |

- A **register (group of flip-flops)** that goes through a prescribed sequence of states upon the application of input pulses is called a *counter* .
- The input pulses may be **clock pulses**, or they may originate from some external source and may occur at a fixed interval of time or at random.
- The sequence of states may follow the binary number sequence or any other sequence of states.
- A counter that follows the binary number sequence is called **a binary counter** .
- An **n -bit binary counter** consists of n flip-flops and can count in binary from 0 through  $2^n - 1$ .

- Counters are available in **two categories**: ripple (or asynchronous) counters and synchronous counters.
- In a **ripple counter**, a flip-flop output transition serves as a source for triggering other flip-flops.
- In other words, the *C* input of some or all flip-flops are triggered, **not by the common clock pulses**, but rather by the transition that occurs in other flip-flop outputs.
- In a **synchronous counter**, the *C* inputs of all flip-flops receive the **common clock**.

# Electronic Principles and Devices

## 3 bit Asynchronous up- counter:

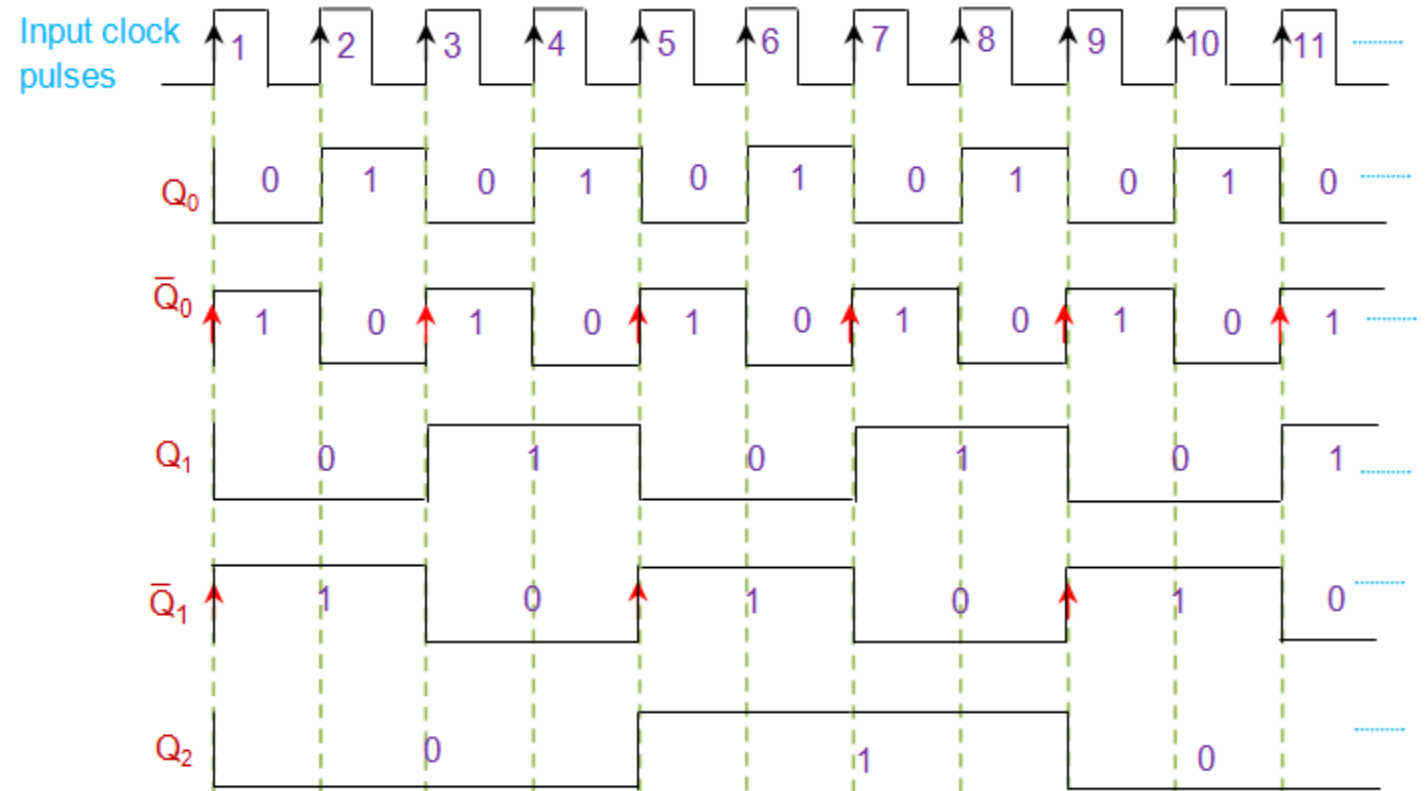


**Circuit  
diagram**

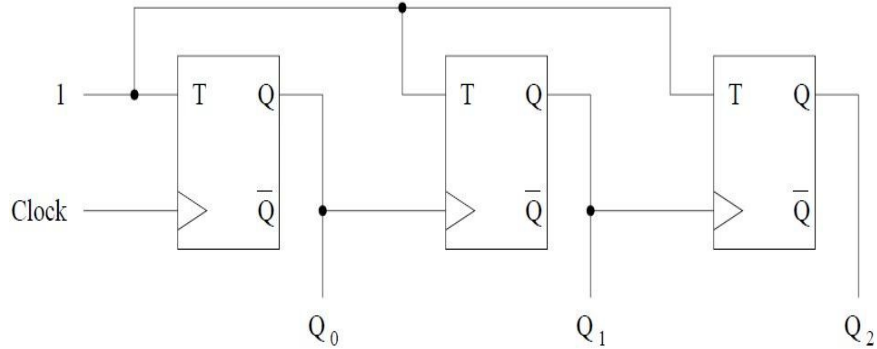
**Truth Table**

| Number of clock pulses | Q <sub>2</sub> (MSB) | Q <sub>1</sub> | Q <sub>0</sub> (LSB) |
|------------------------|----------------------|----------------|----------------------|
| 0                      | 0                    | 0              | 0                    |
| 1                      | 0                    | 0              | 1                    |
| 2                      | 0                    | 1              | 0                    |
| 3                      | 0                    | 1              | 1                    |
| 4                      | 1                    | 0              | 0                    |
| 5                      | 1                    | 0              | 1                    |
| 6                      | 1                    | 1              | 0                    |
| 7                      | 1                    | 1              | 1                    |

**Timing Diagram**



**Figure 4** Timing diagram for 3-bit asynchronous positive edge triggered up-counter

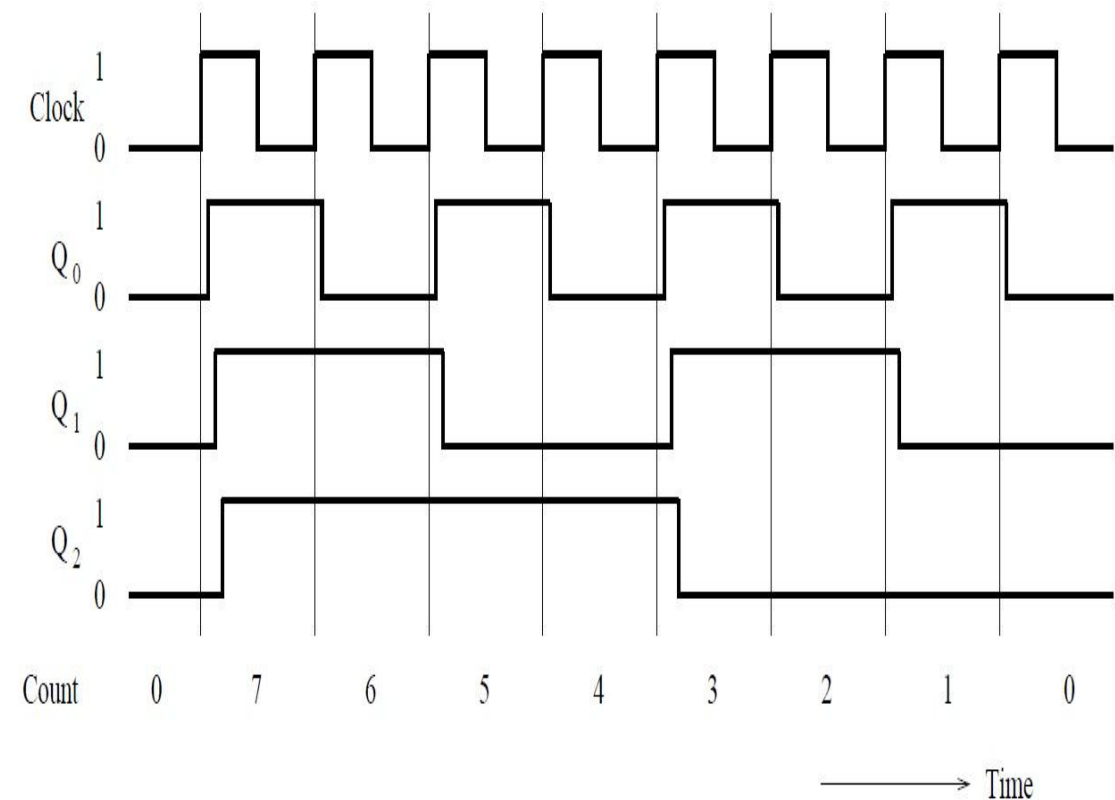


# Circuit diagram

# Truth Table

| Number of clock pulses | $Q_2$ (MSB) | $Q_1$ | $Q_0$ (LSB) |
|------------------------|-------------|-------|-------------|
| 0                      | 0           | 0     | 0           |
| 1                      | 1           | 1     | 1           |
| 2                      | 1           | 1     | 0           |
| 3                      | 1           | 0     | 1           |
| 4                      | 1           | 0     | 0           |
| 5                      | 0           | 1     | 1           |
| 6                      | 0           | 1     | 0           |
| 7                      | 0           | 0     | 1           |

## Timing Diagram





**THANK YOU**

---

Department of Electronics and Communication